



ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ
ΤΜΗΜΑ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΟΠΤΙΚΗ ΑΝΑΠΑΡΑΣΤΑΣΗ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ ΠΟΙΟΤΗΤΑΣ
ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΕΦΟΥΣ ΛΟΓΙΣΜΙΚΟΥ ΜΕ
ΧΡΗΣΗ ΔΙΑΔΙΚΤΥΑΚΩΝ ΕΡΓΑΛΕΙΩΝ

Πτυχιακή Εργασία

του

Μπουκουβάλα Δημήτριου

Επιβλέπων Καθηγητής
Χατζηγεωργίου Αλέξανδρος

Θεσσαλονίκη, 02/2018

ΟΠΤΙΚΗ ΑΝΑΠΑΡΑΣΤΑΣΗ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ ΠΟΙΟΤΗΤΑΣ
ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΕΦΟΥΣ ΛΟΓΙΣΜΙΚΟΥ ΜΕ
ΧΡΗΣΗ ΔΙΑΔΙΚΤΥΑΚΩΝ ΕΡΓΑΛΕΙΩΝ

Μπουκουβάλας Δημήτριος

Πτυχιακή Εργασία

υποβαλλόμενη για τη μερική εκπλήρωση των απαιτήσεων του

ΠΡΟΠΤΥΧΙΑΚΟΥ ΤΙΤΛΟΥ ΣΠΟΥΔΩΝ ΣΤΗΝ ΕΦΑΡΜΟΣΜΕΝΗ ΠΛΗΡΟΦΟΡΙΚΗ

Επιβλέπων Καθηγητής
Χατζηγεωργίου Αλέξανδρος

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή την 02/02/2018

Σατρατζέμη Μαρία
Καθηγήτρια

Ευνόγαλος Στυλιανός
Επίκουρος Καθηγητής

Χατζηγεωργίου Αλέξανδρος
Καθηγητής

.....

.....

.....

Μπουκουβάλας Δημήτριος

.....

Περίληψη

Η ραγδαία αύξηση τόσο της δημοφιλίας των αντικειμενοστρεφών έργων, της πολυπλοκότητας αυτών και των προγραμματιστών που εργάζονται ή συνεισφέρουν σε αυτά, μας τονίζει την σημαντικότητα εύρεσης λύσεων σε προβλήματα όπως είναι η κατανόηση μεγάλων ή παλιών συστημάτων λογισμικού αλλά και ο εύκολος έλεγχος της ποιότητας τους. Αυτό σημαίνει ότι όσο μεγαλώνει ένα έργο τόσο πιο δύσκολο είναι να διατηρηθεί ο έλεγχος και η κατανόηση πάνω σε αυτό από τους υπάρχοντες προγραμματιστές πόσο μάλλον από νέους προγραμματιστές που έρχονται σε επαφή μαζί του είτε για την ανάπτυξη του είτε για την συντήρηση του. Τα παραπάνω προβλήματα αποτελούν πηγές υψηλού κόστους είτε λόγω μειώσεις της παραγωγικότητας είτε λόγω της δυσκολίας για συντήρηση συστημάτων με πολλά προβλήματα κακής σχεδίασης. Με γνώμονα τα παραπάνω η συγκεκριμένη εργασία προσφέρει την δυνατότητα για οπτική αναπαράσταση ενός οποιοδήποτε αντικειμενοστρεφούς έργου ανοιχτού λογισμικού σε μορφή γράφου ώστε να γίνονται ευκολά αντιληπτά τα διάφορα υποσυστήματα και οι σχέσεις μεταξύ τους. Ακόμη γίνεται συνδυαστική αναπαράσταση διαφόρων μετρικών ανάλυσης ποιότητας, κλιμακώνοντας το μέγεθός των κόμβων με βάση επιλεγμένων μετρικών, φιλτράροντας τούς με βάση συγκεκριμένες τιμές, είτε επιλέγοντας ποιοι κομβοί θα εμφανιστούν ανάλογα με το είδος της σχέσεις τους με τον επιλεγμένο κόμβο. Χρησιμοποιώντας τις παραπάνω δυνατότητες και με βάση τις αρχές της αντικειμενοστρεφούς σχεδίασης αναλύονται ακόμη στρατηγικές εντοπισμού προβλημάτων κακής σχεδίασης.

Λέξεις Κλειδιά: Οπτική Αναπαράσταση, Γράφος, Αντικειμενοστρεφής Σχεδίαση, Μετρικές Ποιότητας

Abstract

Undoubtedly there is a rapid increase in the popularity of object-oriented projects along with their complexity, size and the programmers that work or contribute on them. This tendency emphasizes the importance of finding solutions in problems such as the understanding of large or legacy systems and the assurance of the project's quality. This means that as the project grows, the more difficult it is for the developers to maintain the control and the understanding on it. The above problems result on high expenses either due to the lack of productivity or due to the increased effort that is expected for maintaining a system with many poor design problems. Driven by these problems and needs the main goal of this bachelor thesis is to offer a graph visualization of any object-oriented open source project in order for the various subsystems and the relations amongst them to be easily perceived. Furthermore, the representation includes various quality metrics for the project analysis, and you could scale the nodes according the selected metrics, filter the nodes based a specific value or even choose which nodes should appear based on their relation with a selected node. There are also detecting strategies that we could follow for bringing to the surface design problems, based on the above capabilities and the application of object-oriented principles.

Keywords: Graph Visualization, Object Oriented Design, Quality Metrics

Ευχαριστίες

Θα ήθελα να ευχαριστήσω τον επιβλέποντα καθηγητή κ.Χατζηγεωργίου Αλέξανδρο και τον επιστημονικό συνεργάτη του τμήματος κ.Χαϊκάλη Θωδωρή για την πολύτιμη βοήθεια και καθοδήγηση τους καθ' όλη την διάρκεια της εργασίας.

Ακόμη θα ήθελα να ευχαριστήσω την οικογένεια μου που με στήριξε όλα αυτά τα χρόνια και να αφιερώσω αυτή την εργασία στον πατέρα μου.

Περιεχόμενα

1 Εισαγωγή	1
1.1 Πρόβλημα – Σημαντικότητα του θέματος – Σκοπός – Στόχοι	1
1.2 Συνεισφορά	2
1.3 Διάρθρωση της μελέτης	2
2 Βιβλιογραφική Επισκόπηση – Θεωρητικό Υπόβαθρο	3
2.1 Γράφοι	3
2.1.1 Θεωρία Γράφων	3
2.1.2 Ο Γράφος Στην Περίπτωση Μας.	4
2.2 Αρχές Αντικειμενοστρεφούς Σχεδίασης	5
2.3 Προβλήματα και Συμπτώματα κακής αντικειμενοστρεφούς σχεδίασης	6
2.3.1 Προβλήματα	7
2.3.2 Συμπτώματα – Code Smells	8
2.4 Μετρικές	9
2.4.1 Μετρικές Γράφου	9
2.4.2 Μετρικές Πηγαίου Κώδικα	10
2.5 Τεχνολογίες Που Χρησιμοποιήθηκαν	11
2.5.1 Εμφανισιακό μέρος	11
2.5.2 Λειτουργικό κομμάτι και Visualization	11
3 Απαιτήσεις Εφαρμογής	14
3.1 Πρόβλημα	14
3.2 Που βασίστηκε η εφαρμογή	15
3.3 Επέκταση	16
3.4 Σύγκριση Παρόμοιων Εργαλείων	17
3.4.1 Γενικά	17
3.4.2 Ερευνητικά Εργαλεία που Ξεχωρίζουν	20
4 Σχεδίαση και υλοποίηση εφαρμογής	24
4.1 Τεχνικό κομμάτι εργασίας	24
4.1.1 Επιλογή Project και Γενιάς	24
4.1.2 Γραφική αναπαράσταση γράφου	25
4.1.3 Μπάρα ρυθμίσεων	28

5 Παρουσίαση της εφαρμογής	30
5.1 Εξαγωγή Πληροφοριών μέσω Visualization	30
5.1.1 Εντοπισμός God Class	30
5.1.2 Εντοπισμός Brain Classes	36
5.1.3 Εντοπισμός Data Classes	38
5.1.4 Εντοπισμός Feature Envy	39
5.2 Περίπτωση Χρήσης	40
6 Επίλογος	47
6.1 Συμπεράσματα	47
6.2 Μελλοντική επέκταση	47

Κατάλογος Εικόνων

Εικόνα 1 : Εμφάνιση Δεδομένων SEAgile	15
Εικόνα 2 : Correlation Analysis SEAgile	16
Εικόνα 3 : UML class diagram.....	18
Εικόνα 4 : Μεγάλης κλίμακας UML διάγραμμα.....	18
Εικόνα 5 : Lanza Visualization	20
Εικόνα 6 : Επιλογή περιοχής στο έργο ArgoUML.....	21
Εικόνα 7 : Code smells και City Visualization	22
Εικόνα 8 : Πόλη με business προβλήματα.....	23
Εικόνα 9: Ρυθμίσεις Scaling	26
Εικόνα 10 : Τονισμός Πολύπλοκων Κλάσεων.....	30
Εικόνα 11 : Τονισμός κλάσεων με υψηλές τιμές ATFD	31
Εικόνα 12 Σχέση μεταξύ NOPA και God Classes.	32
Εικόνα 13 : Συνεκτικότητα	32
Εικόνα 14 Clustering Coefficient and God Classes	33
Εικόνα 15 Coupling και God Classes.....	33
Εικόνα 16 : Lines of Code.....	36
Εικόνα 17 : WMC and Brain Classes	36
Εικόνα 18 : ATFD και Brain Classes	37
Εικόνα 19 : NOPA και Data Classes	38
Εικόνα 20 : NOAM και Data Classes.....	38
Εικόνα 21 : Συνδυασμός Scale και Filter για Feature Envy	39
Εικόνα 22 : Αρχική σελίδα για αναζήτηση project	40
Εικόνα 23 : Επιλεγμένο project και γενιά.....	41
Εικόνα 24 : Γραφική Αναπαράσταση Επιλεγμένου Project και Μπάρα Ρυθμίσεων	41
Εικόνα 25 : Scale με επιλεγμένη μετρική Out Degree	43
Εικόνα 26 : Scale με επιλεγμένη μετρική Betweenness Centrality.....	44
Εικόνα 27 : Φίλτρο για την μετρική ATFD και scale με WMC	45
Εικόνα 28 : Εμφάνιση μόνο των εισερχόμενων ακμών.....	46
Εικόνα 29 : Εμφάνιση μόνο των εξερχόμενων ακμών	46

1 Εισαγωγή

1.1 Σημαντικότητα του θέματος – Σκοπός

Η ανάπτυξη λογισμικού είναι μια πολύπλοκη και δύσκολη διαδικασία που απαιτεί μεγάλη προσπάθεια τόσο κατά την αρχική φάση της σχεδίασης και ανάπτυξης αλλά ακόμη περισσότερο κατά την συντήρηση του. Υπάρχουν πολλά προβλήματα που μπορούν να προκύψουν καθ' όλη την διάρκεια και αυτά γίνονται ακόμα εντονότερα όσο μεγαλώνει το μέγεθος του έργου και η πολυπλοκότητα του. Κύριο πρόβλημα αποτελεί η δυσκολία των προγραμματιστών να κατανοήσουν την δομή του έργου είτε επειδή είναι καινούργιοι σε ένα υπάρχον παλιό έργο είτε επειδή το έργο τούς έχει κλιμακωθεί σε τέτοιο βαθμό και με τέτοιον τρόπο οπού έχουν χάσει τον έλεγχο του. Ακόμη ένα πρόβλημα οπού μερικές φορές πηγάζει από την παραπάνω έλλειψη κατανόησης είναι η απόκλιση από τις αρχές αντικειμενοστραφούς σχεδίασης, δημιουργώντας έτσι προβλήματα κακής σχεδίασης (Code Smells) οπού στην συνέχεια είναι δύσκολο να εντοπιστούν. Επειδή τέτοιου είδους προβλήματα συνθέτουν για μία εταιρία πηγες κόστους και χαμηλής παραγωγικότητας χρειάζεται ένας εύκολος και γρήγορος τρόπος για να αντιμετωπιστούν. Με γνώμονα την εκ φύσεως κλίση του ανθρώπου να βγάζει πιο ευκολά συμπεράσματα για μεγάλες και πολύπλοκες έννοιες με μία εικόνα, επιλέχθηκε ως τρόπος εξαγωγής πληροφοριών για το εξεταζόμενο έργο ένα δίκτυο συσχετίσεων ή πιο απλά γράφος. Επειδή όμως για την κατανόηση, τον εντοπισμό προβλημάτων σχεδίασης και γενικότερα για την εκτίμηση της ποιότητας του έργου δεν αρκεί μόνο αυτό, ο γράφος συνδυάστηκε με την γραφική απεικόνιση σημαντικών μετρικών ανάλυσης.

1.2 Συνεισφορά

Η απεικόνιση ενός συστήματος λογισμικού βοηθά σημαντικά στην κατανόηση της δομής του και στον εντοπισμό αρχιτεκτονικών ανωμαλιών. Η παρούσα εργασία έρχεται να συμπληρώσει τη λειτουργικότητα της πλατφόρμας Seagle προσφέροντας της δυνατότητα απεικόνισης σε μορφή γράφου των αντικειμενοστρεφών συστημάτων που έχουν αναλυθεί. Η ευκολία χρήσης δίνει τη δυνατότητα οπτικοποίησης οποιουδήποτε αντικειμενοστρεφούς έργου αρκεί να βρίσκεται σε διαδικτυακό αποθετήριο git με ελεύθερη πρόσβαση.

Επιπλέον, γίνεται δυνατός ο συνδυασμός μετρικών γράφου και πηγαίου κώδικα μαζί με την γραφική αναπαράσταση του εξεταζόμενου έργου ως γράφου. Πιο συγκεκριμένα το μέγεθος των κόμβων του γράφου μπορεί να κλιμακωθεί σύμφωνα με οποιαδήποτε επιλεγμένη μετρική ώστε να τονιστούν πιθανά προβλήματα. Ακόμη υπάρχει η δυνατότητα να φιλτραριστούν οι κόμβοι του γράφου με βάση την σύγκρισή μεταξύ της τιμής τους σε μία μετρική και της τιμής που εισάγουμε εμείς. Οι δύο παραπάνω δυνατότητες μπορούν να συνδυαστούν.

Για να διευκολυνθεί η διαδικασία της απόκτησης πληροφορίας και της ανάλυσης δίνεται η δυνατότητα να επιλεγθεί διαφορετικό συμβάν κατά την επιλογή ενός κόμβου ώστε να καλυφθούν οι διαφορετικές απαιτήσεις. Τέλος γίνεται χρωματική ομαδοποίηση των κόμβων με βάση το πακέτο της κλάσης που αναπαριστούν ώστε να εξάγουμε ευκολά μια επιπλέον πληροφορία για τις σχέσεις των κλάσεων.

1.3 Διάρθρωση της μελέτης

Στο Κεφάλαιο 2 γίνεται μία παρουσίαση του θεωρητικού υποβάθρου που βασίστηκε η εργασία. Στο Κεφάλαιο 3 αναλύεται το πρόβλημα που επιχειρείτε να λυθεί , το που βασίστηκε η λύση , τι καινούργιο προστέθηκε και παρουσιάζονται και αλλά παρόμοια εργαλεία. Στο Κεφάλαιο 4 παρουσιάζετε η εργασία από τεχνική σκοπιά. Στο Κεφάλαιο 5 αναλύονται τρόποι για τον εντοπισμό προβλημάτων κακής σχεδίασης. Στο Κεφάλαιο 6 δίνεται η σύνοψη της εργασίας και σκέψεις για μελλοντική επέκταση.

2 Βιβλιογραφική Επισκόπηση – Θεωρητικό Υπόβαθρο

2.1 Γράφοι

2.1.1 Θεωρία Γράφων

Ο γράφος σύμφωνα με τα διακριτά μαθηματικά αποτελείται από ένα σύνολο αντικειμένων τα οποία συνδέονται μεταξύ τους με κάποια σχέση. Πιο συγκεκριμένα αποτελείται από κόμβους (nodes) ή κορυφές (vertices) όπου οι σχέσεις μεταξύ τους ονομάζονται είτε ακμές (edges) είτε τόξα (arcs).

Πιθανολογείται ότι η πρώτη χρήση της θεωρίας γράφου συνέβη τον 18^ο αιώνα, όταν ο Leonhardt Euler έλυσε το πρόβλημα των 7 γεφυρών της πόλης Königsberg σχεδιάζοντας το ως πρόβλημα γράφου [1]. Σήμερα οι εφαρμογές της θεωρίας γράφων αυξάνονται συνεχώς καθώς χρησιμοποιείται σε διάφορους τομείς της ζωής μας όπως στη βιοχημεία, στην κοινωνιολογία, στα δίκτυα και συστήματα τηλεπικοινωνιών, στους χάρτες για την επιλογή διαδρομών (GPS), στην ανάλυση προγραμμάτων (flowcharts) και σε αμέτρητες περιπτώσεις στην επιστήμη των υπολογιστών.

Οι γράφοι χωρίζονται σε δύο μεγάλες κατηγορίες, τους μη-κατευθυνόμενους και τους κατευθυνόμενους. Στο **μη-κατευθυνόμενο γράφο** οι ακμές που συνδέουν δυο κόμβους δεν έχουν προσανατολισμό, είναι αμφίδρομες. Για παράδειγμα η ακμή (α, β) που ενώνει τον κόμβο A και τον κόμβο B είναι ίδια με την ακμή (β, α) . Αντίθετα στους **κατευθυνόμενους γράφους** η γραφική αναπαράσταση των σχέσεων γίνεται από βέλη. Για παράδειγμα για ένα βέλος (α, β) θεωρείτε ότι ξεκινάει από τον κόμβο A και έχει κατεύθυνση τον κόμβο B. Το β ονομάζεται η κεφαλή του τόξου και το α η ουρά του. Ακόμη μπορούμε να πούμε το β ότι είναι ο άμεσος απόγονος του α , και το α είναι ο άμεσος πρόγονος του β . Τα τόξα (α, β) και (β, α) δεν είναι ίδια όπως στην περίπτωση του μη-κατευθυνόμενου γράφου αλλά το ένα είναι το αντίστροφο του αλλού.

Αν για κάθε ζευγάρι κόμβων ενός γράφου υπάρχει διαδρομή που τους συνδέει τότε ο γράφος είναι **συνεκτικός** και συγκεκριμένα ο μη-κατευθυνόμενος γράφος ονομάζεται ελαφρά συνεκτικός και ο κατευθυνόμενος γράφος ονομάζεται ισχυρά συνεκτικός. Ο γράφος που ικανοποιεί την παραπάνω περίπτωση ονομάζεται και **πλήρης**, αν περιέχει

λίγες ακμές αραιός ή αν περιέχει πολλές πυκνός. Η **διάμετρος** ενός γράφου είναι η μέγιστη απόσταση μεταξύ δυο οποιοδήποτε κόμβων του. Ακόμη για κάθε κόμβο υπάρχουν οι **γειτονικοί κόμβοι** που συνδέονται άμεσα μαζί του και ο **βαθμός (degree)** κορυφής, ο οποίος είναι το πλήθος των ακμών τις συγκεκριμένης κορυφής. Στους κατευθυνόμενους γράφους ο βαθμός (degree) κορυφής χωρίζεται σε in-degree και out-degree, όπου in-degree μετράει τις ακμές που καταλήγουν στον κόμβο από γειτονικούς κόμβους, ενώ out-degree είναι οι ακμές που ξεκινάνε από τον συγκεκριμένο κόμβο και κατευθύνονται προς γειτονικούς.

Γενικά δεν θα πρέπει να συγχέεται ο σχεδιασμός ενός γράφου με τον ίδιο τον γραφό που είναι μία μη-εικονιζόμενη δομή, καθώς υπάρχουν πολύ τρόποι για να διαταχθούν τα δεδομένα που θέλουμε να απεικονιστούν. Αυτό όμως που δεν θα πρέπει να αλλάζει είναι το πλήθος των κόμβων και οι σχέσεις που υπάρχουν μεταξύ τους. Αναλόγως το είδος του προβλήματος ίσως κάποιες διατάξεις είναι καλύτερες από κάποιες άλλες.

2.1.2 Ο Γράφος Στην Περίπτωση Μας.

Στη περίπτωση μας χρησιμοποιήθηκε ένας κατευθυνόμενος γράφος με τους κόμβους του να

αναπαριστούν τις java κλάσεις του έργου που έχει αναλυθεί. Οι σχέσεις μεταξύ των κλάσεων βασίζονται στο Law of Demeter [2]. Πιο συγκεκριμένα υπάρχει ένα βέλος από τον κόμβο A προς τον κόμβο B όταν:

- Η κλάση A περιέχει ιδιότητα τύπου κλάσης B.
- Μέθοδος της κλάσης A περιέχει μια τοπική μεταβλητή τύπου B.
- Μέθοδος της κλάσης A περιέχει παράμετρο τύπου B.
- Μέθοδος της κλάσης A επιστρέφει αντικείμενο τύπου B.
- Μέθοδος της κλάσης A χρησιμοποιεί δημόσιο αντικείμενο τύπου B.

Κατά τον σχεδιασμό του γράφου οι κόμβοι ομαδοποιούνται και παίρνουν την θέση τους στο χώρο ανάλογα των σχέσεων που υπάρχουν μεταξύ τους. Απεικονίζονται ως πολύχρωμες τελείες όπου το χρώμα τους εξαρτάται από το πακέτο στο οποίο βρίσκεται η αντίστοιχη κλάση, ώστε να γίνεται ευκολά αντιληπτό ποιες είναι κλάσεις ιδίου πακέτου. Εκτός από τελείες οι κόμβοι μπορούν να έχουν και διαφορετικό σχήμα ανάλογα με το αν η κλάση έχει χαρακτηριστεί ως ένα από τα code smells. Συγκεκριμένα αν η κλάση είναι

ορισμένη ως God Class τότε ο αντίστοιχος κόμβος έχει σχήμα αστεριού , αν είναι ορισμένη ως Data Class ο κόμβος έχει σχήμα ρόμβου και τέλος αν εμφανίζει Feature Envy ο κόμβος έχει σχήμα τετραγώνου. Το μέγεθος κάθε κόμβου εξαρτάται από την τιμή του στην εκάστοτε μετρική που έχει επιλεχθεί για την κλιμάκωση (scaling).

2.2 Αρχές Αντικειμενοστρεφούς Σχεδίασης

Υπάρχουν αρκετές αρχές όπου δίνουν κάποιες κατευθυντήριες γραμμές για την σωστή ανάπτυξη λογισμικού [3] [4]. Η συγκεκριμένη εργασία όμως επικεντρώνεται στις αρχές που εφαρμόζονται στην αντικειμενοστραφή σχεδίαση και ειδικά σε αυτές που έχουν νόημα να μελετηθούν από το επίπεδο του γράφου των κλάσεων. Παρακάτω γίνεται μια παρουσίαση τέτοιων αρχών :

- **Αρχή της ενσωμάτωσης (encapsulation):** Μπορούμε να φανταστούμε την κλάση στον αντικειμενοστρεφή προγραμματισμό σαν μια κάψουλα όπου εμπεριέχει τα δεδομένα (κατάσταση) και την λειτουργικότητα (συμπεριφορά) της μαζί. Τα δεδομένα αποθηκεύονται στις ιδιότητες της κλάσης , οι οποίες σύμφωνα με την αρχή της ενσωμάτωσης θα πρέπει να είναι ιδιωτικές. Ο λόγος για αυτό είναι ότι δεν θα πρέπει να μπορεί οποιοσδήποτε βρίσκεται εκτός κλάσης να προσπελάσει ή να μετατρέψει απευθείας τις ιδιότητές της. Η δυνατότητα αυτή δίνεται μέσω των δημόσιων μεθόδων της όπου μπορούν να έχουν καλύτερο έλεγχο στην διαδικασία της προσπέλασης και της μετατροπής .
- **Αρχή της μοναδικής αρμοδιότητας (single responsibility):** Σύμφωνα με τον Robert C. Martin “Μια κλάση πρέπει να έχει μόνο έναν λόγο για να αλλάξει ” [5]. Η παραπάνω πρόταση σημαίνει ότι το περιεχόμενο μιας κλάσης θα πρέπει να έχει συνοχή και όλα τα επιμέρους μέρη της κλάσης να δουλεύουν για έναν κοινό σκοπό. Καθώς αν μία κλάση έχει παραπάνω από μία αρμοδιότητες τότε θα έχει και αντίστοιχους λόγους για να αλλάξει.
- **Αρχή της υψηλής συνεκτικότητας (High cohesion):** Η αρχή της μοναδικής αρμοδιότητας είναι συνυφασμένη με την έννοια της συνεκτικότητας όπου δηλώνει σε τί βαθμό η κλάση έχει έναν και καλά ορισμένο σκοπό. Όσο μεγαλύτερος ο βαθμός της συνεκτικότητας τόσο καλύτερη και η αντικειμενοστραφής σχεδίαση. Τα θετικά της υψηλής συνεκτικότητας είναι ότι α) οι κλάσεις που έχουν υψηλή συνεκτικότητα είναι

πιο εύκολο να συντηρηθούν και αλλάζουν πιο σπάνια. β) μπορούν να χρησιμοποιηθούν πιο συχνά από άλλες κλάσεις.

- **Μεγιστοποίηση Fan-in:** Με τον όρο Fan-in εννοούμε το πλήθος των κλάσεων που χρησιμοποιούν την συγκεκριμένη κλάση. Για μία καλή αντικειμενοστρεφή σχεδίαση επιδιώκεται η αύξηση της Fan-in τιμής καθώς σημαίνει ότι η συγκεκριμένη κλάση έχει υψηλή συνεκτικότητα και μοναδική αρμοδιότητα. Γενικά είναι προτιμότερο μία λειτουργία που χρειάζεται συχνά να βρίσκεται συγκεντρωμένη σε μία κλάση και να καλείται από τις υπόλοιπες παρά να υλοποιείτε σε κάθε μια από τις κλάσεις που την χρειάζονται.
- **Ελαχιστοποίηση Fan-out:** Ο όρος Fan-out χρησιμοποιείται για να δείξει πόσες άλλες κλάσεις χρησιμοποιεί άμεσα η εξεταζόμενη κλάση. Στόχος μας είναι να περιορίσουμε την συγκεκριμένη τιμή καθώς μια υψηλή Fan-out τιμή συνεπάγεται ότι η κλάση έχει πολλές εξαρτήσεις αρά και χαμηλή συνεκτικότητα.
- **Αρχή της χαμηλής σύζευξης (loose coupling):** Αρχικά θεωρούμε ότι υπάρχει σύζευξη μεταξύ δυο κλάσεων όταν έστω η μια κλάση γνωρίζει για την άλλη, αυτό μπορεί να συμβεί αν για παράδειγμα η μία καλεί κάποια δημόσια μέθοδο της άλλης, έχει δηλωμένες ιδιότητες της ή χρησιμοποιεί παραμέτρου της. Όσο χαμηλότερη η τιμή της σύζευξης τόσο μικρότερη και η αλληλεξάρτηση μεταξύ δύο κλάσεων, κάτι που διευκολύνει την συντήρηση του λογισμικού και τις αλλαγές σε αυτό. Είναι εύκολο να φανταστούμε αν σε ένα σύστημα με υψηλή σύζευξη θελήσουμε να κάνουμε μια αλλαγή σε μία κλάση η πιθανότητα να χρειαστεί να αλλάξουμε και τις υπόλοιπες κλάσεις είναι πολύ υψηλή.

2.3 Προβλήματα και Συμπτώματα κακής αντικειμενοστρεφούς σχεδίασης

Η αρχική φάση της ανάπτυξης λογισμικού συνήθως είναι μια περίοδος χάριτος καθώς όλα σχεδιάζονται από την αρχή χωρίς πολλούς περιορισμούς και προβλήματα. Στο μυαλό όλων κατά την σχεδίαση υπάρχουν οι καλύτερες πρακτικές που μπορούν να χρησιμοποιηθούν, ιδιαίτερα χαρακτηριστικά να προστεθούν αλλά και υποσχέσεις για μία κοινή φιλοσοφία και σταθερές που πρέπει να τηρηθούν. Κάποιες ομάδες καταφέρνουν και διατηρούν την πορεία τους καθ' όλη την διάρκεια της ανάπτυξης, τις περισσότερες φορές όμως κάτι τέτοιο δεν συμβαίνει. Οι μικρές αποκλίσεις από το αρχικό σχεδιασμό μαζί με εκπτώσεις στην ποιότητα συνήθως για να κερδιστεί χρόνος βραχυπρόθεσμα οδηγούν σίγα

και σταθερά σε βέβαια προβλήματα. Σύμφωνα με τον Robert Martin κάποια από αυτά τα προβλήματα είναι τα παρακάτω [6].

2.3.1 Προβλήματα

Δυσκαμψία (Rigidity): Θεωρούμε ότι υπάρχει δυσκαμψία σε ένα λογισμικό όταν μια αλλαγή σε ένα κομμάτι του προγράμματος προκαλεί αλυσιδωτές αλλαγές και σε πολλά άλλα σημεία του προγράμματος. Αυτή η συμπεριφορά δυσκολεύει τους προγραμματιστές να εκτιμήσουν τον χρόνο που χρειάζονται μέχρι και για μία φαινομενικά απλή αλλαγή στον κώδικα. Οι project managers με την σειρά τους επιτρέπουν στους προγραμματιστές να προβαίνουν μόνο σε αλλαγές που είναι απολύτως απαραίτητο να συμβούν καθώς φοβούνται ότι αν τους δώσουν μεγαλύτερη ελευθερία ίσως "χαθούν" στην προσπάθεια τους να αλλάξουν κάτι που οδηγεί και σε πολλές άλλες αλλαγές.

Ευθραυστότητα (Fragility): Πρόκειται για ένα παρόμοιο σύμπτωμα με την Δυσκαμψία καθώς σχετίζεται με το πρόβλημα ότι μία αλλαγή μπορεί να προκαλέσει ένα πρόβλημα σε ένα άκυρο σημείο από εκεί που συνέβη η αρχική αλλαγή. Συστήματα με υψηλή ευθραυστότητα είναι σχεδόν αδύνατο να συντηρηθούν καθώς η πιθανότητα μια αλλαγή να δημιουργήσει κάπου αλλού πρόβλημα αγγίζει την βεβαιότητα. Το αποτέλεσμα είναι οι προγραμματιστές να χάνουν την αξιοπιστία τους απέναντι στον project manager αλλά και στους πελάτες.

Ακινησία (Immobility): Όταν είναι δύσκολο να επαναχρησιμοποιηθεί ένα κομμάτι λογισμικού από το ίδιο πρόγραμμα ή από διαφορετικό. Για παράδειγμα όταν θα μπορούσε να γίνει χρήση και σε άλλα σημεία του λογισμικού ενός τμήματος κώδικα αλλά λόγω της σχεδίασης του και της αυξημένης αλληλεξάρτησης του με άλλα τμήματα, η προσπάθεια που θα χρειαστεί για να επαναχρησιμοποιηθεί ξεπερνά την προσπάθεια που χρειάζεται για να φτιαχτεί ξανά από την αρχή.

Έλλειψη ρευστότητας (Viscosity): Οι προγραμματιστές έχουν δύο τρόπους για να κάνουν μία αλλαγή α) ο τρόπος οπου είναι σύμφωνος και διατηρεί τον υπάρχων σχεδιασμό β) με κάποιο τέχνασμα (hack). Όταν για την πραγματοποίηση μια αλλαγής ο δεύτερος και λανθασμένος τρόπος είναι πιο εύκολος από τον πρώτο και σωστό τρόπο τότε θεωρούμε ότι υπάρχει έλλειψη ρευστότητας.

2.3.2 Συμπτώματα – Code Smells

Τα Code smells θα μπορούσαν να χαρακτηριστούν ως τα συμπτώματα ενός προβλήματος και όχι ως το ίδιο το πρόβλημα. Οι Martin Fowler και Kent Beck θεωρούν τα code smells ως ενδείξεις όπου είτε άμεσα είτε με πιο βαθύ ψάξιμο οδηγούν σε πολύ σημαντικά και ενδιαφέροντα προβλήματα [4] .

Code Smells βρίσκονται στο επίπεδο εντός της κλάσεως αλλά και στο επίπεδο μεταξύ των κλάσεων. Παρακάτω θα μελετηθούν στο δεύτερο επίπεδο και σύμφωνα με το έργο του Michele Lanza [7].

- **God Class:** Πρώτη φορά αναφέρθηκε από τον Riel το 1996 [3]. Πρόκειται συνήθως για μια μεγάλη κλάση όπου τείνει να συγκεντρώνει όλη την λογική του συστήματος και να χρησιμοποιεί τις υπόλοιπες κλάσεις για ασήμαντες λειτουργίες ή μόνο για δεδομένα. Η συγκεκριμένη τύπου κλάση αντιτίθεται με την φιλοσοφία του αντικειμενοστρεφή προγραμματισμού, ότι η ευφυΐα του συστήματος θα πρέπει να ισοκατανέμεται μεταξύ του υψηλού επιπέδου κλάσεων. Ακόμη γίνεται εύκολα αντιληπτό ότι δεν ακολουθεί την αρχή της μοναδικής αρμοδιότητας.
- **Brain Class:** Είναι παρόμοιου τύπου κλάση με την God Class αλλά συνήθως υπάρχει λίγο μεγαλύτερος βαθμός συνεκτικότητας και ίσως δεν είναι τόσο καταχρηστική με την χρήση δεδομένων από άλλες κλάσεις. Εξαιτίας αυτών των διαφορών υπάρχει η περίπτωση να μη θεωρηθεί God class και να μην της δοθεί η κατάλληλη προσοχή, κάτι που θα ήταν λάθος καθώς μοιράζεται τις ίδιες αρνητικές συνέπειες με τις God classes.
- **Data Class:** Είναι μια απλή κλάση όπου περιέχει μόνο ιδιότητες και τους αντίστοιχους Getters χωρίς να γίνετε κάποια υλοποίηση που να επεκτείνει την λογική του συστήματος. Είναι δείγμα κακής σχεδίασης καθώς διαχωρίζονται τα δεδομένα από την λογική.
- **Feature Envy:** Σχετίζεται με τις μεθόδους όπου δείχνουν αυξημένο ενδιαφέρον για δεδομένα ξένων κλάσεων παρά τον δικό τους. Οι συγκεκριμένες μέθοδοι χρησιμοποιούν είτε απευθείας είτε μέσω των Getter μεθόδων τα δεδομένα της ξένης κλάσης. Το συγκεκριμένο code smell είναι ένα σημάδι ότι ίσως η μέθοδος θα έπρεπε να μετακινηθεί ώστε να είναι μαζί με τα δεδομένα που χρησιμοποιεί .

2.4 Μετρικές

Οι μετρικές στην ανάπτυξη λογισμικού αποτελούν ένα σημαντικό εργαλείο για την μέτρηση της ποιότητας του αλλά και της ύπαρξης σωστής σχεδίασης. Πιο συγκεκριμένα βοηθάνε στην κατανόηση και στον έλεγχο μεγάλων και πολύπλοκων συστημάτων καθώς μέσω αυτών μπορούν να εντοπιστούν πιθανά σχεδιαστικά προβλήματα. Με αλλά λόγια είναι η αρχική ένδειξη για βαθύτερη ανάλυση και αλλαγή σε ένα σύστημα λογισμικού. Παρακάτω εξετάζονται οι μετρικές γράφου και πηγαίου κώδικα.

2.4.1 Μετρικές Γράφου

- **Αριθμός Κόμβων:** Ο αριθμός των κόμβων ενός γράφου για μια συγκεκριμένη γενιά του συστήματος.
- **Αριθμός Εσωτερικών Ακμών:** Ο αριθμός των ακμών που έχουν ως κατεύθυνσή τον συγκεκριμένο κόμβο.
- **Αριθμός Εξωτερικών Ακμών:** Ο αριθμός των ακμών που ξεκινάνε από τον συγκεκριμένο κόμβο.
- **Betweenness Centrality:** Αποτελεί ένδειξη για την εκκεντρότητα ενός κόμβου στο ευρύτερο δίκτυο. Η τιμή της είναι ίση με τον αριθμό των συντομότερων μονοπατιών τα οποία περνάνε από το συγκεκριμένο κόμβο. Κόμβοι με υψηλή τιμή έχουν σημαντική επιρροή στο δίκτυο καθώς έχουν έλεγχο στις πληροφορίες που περνάνε μέσω αυτών. Ακόμη η αφαίρεση των συγκεκριμένων κόμβων θα δημιουργούσε προβλήματα στην υπάρχουσα επικοινωνία των υπολοίπων κόμβων
- **Clustering Coefficient:** Ορίζεται ως η πιθανότητα σχετικά με το πόσο πιθανό είναι οι “γείτονες” του κόμβου A να είναι και “γείτονες” μεταξύ τους. Στις τιμές κοντά στο 1 λέμε ότι σχηματίζεται κλίκα.

$$CC(v) = (2 * N_v) / (K_v * (K_v - 1))$$
, όπου N_v είναι ο αριθμός των συνδέσμων μεταξύ των γειτόνων του κόμβου v και K_v είναι ο αριθμός των γειτόνων για τον κόμβο v .

2.4.2 Μετρικές Πηγαίου Κώδικα

- **Access To Foreign Data (ATFD):** Αναπαριστά τον αριθμό των κλάσεων όπου η συγκεκριμένη κλάση χρησιμοποιεί τις ιδιότητες τους είτε άμεσα είτε μέσω των getter μεθόδων τους.
- **Tight Class Cohesion (TCC):** Είναι ο αριθμός των άμεσα συνδεδεμένων μεθόδων μίας κλάσης. Λέμε ότι δυο κλάσεις είναι συνδεδεμένες όταν προσπελάζουν την ίδια ιδιότητα της κλάσης [8].
- **Weighted Method Complexity (WMC):** Είναι το άθροισμα της πολυπλοκότητας όλων των μεθόδων της κλάσης. Αποτελεί ένδειξη για το μέγεθος της προσπάθειας που χρειάζεται για να αναπτυχθεί αλλά και για να συντηρηθεί μια κλάση.
- **Coupling Between Objects (CBO):** Είναι το πλήθος των συσχετίσεων μιας κλάσης με άλλες κλάσεις. Συσχέτιση μεταξύ δυο κλάσεων A, B προκύπτει όταν στην κλάση B υπάρχει ιδιότητα τύπου A ή όταν τουλάχιστον μια μέθοδος στη B χρησιμοποιεί άλλες μεθόδους, μεταβλητές, παραμέτρους τύπου A ή έχει επιστρεφόμενη μεταβλητή τύπου A. Ακόμη σύζευξη υπάρχει όταν υπάρχει κληρονομικότητα μεταξύ των A και B.
- **Lack of Cohesion in Methods 2 (LCOM2):** Η πρώτη έκδοση της συγκεκριμένης μετρικής εισάχθηκε από τους Chidamber και Kemerer το 1994 [9]. Λόγω διαφόρων προβλημάτων που υπήρξαν κατά το πέρασμα των χρόνων ο εναλλακτικός τρόπος υπολογισμού LCOM2 έκανε την εμφάνιση του από τον Brian Henderson-Sellers [10]. Η μετρική LCOM2 υπολογίζει την συνεκτικότητα μίας κλάσης ως το ποσοστό των μεθόδων που δεν χρησιμοποιούν κάποια ιδιότητα δια των συνολικών ιδιοτήτων της κλάσης.
- **Lines of Code of a class (LOC):** Οι γραμμές του κώδικα για την κάθε κλάση.
- **Number of Accessor Methods of a class (NOAM):** Είναι ο αριθμός των getters που βρίσκονται σε μία κλάση και δεν έχουν κληρονομηθεί.
- **Number of Methods for each class (NOM):** Το πλήθος των μεθόδων που βρίσκεται σε κάθε κλάση.
- **Number of Fields (NOF):** Το πλήθος των πεδίων για κάθε κλάση.

- **Number of Public Attributes (NOPA):** Το πλήθος των δημόσιων ιδιοτήτων για κάθε κλάση.
- **Weight of Class:** Το πλήθος των ‘λειτουργικών’ δημόσιων μεθόδων δια τον συνολικό αριθμό των δημοσίων μεθόδων. Με την έννοια ‘λειτουργική’ μέθοδος εννοούμε όλες τις μεθόδους εκτός Getters – Setters και Κατασκευαστή (Constructor). Η συγκεκριμένη μετρική προσπαθεί να εντοπίσει αν μια κλάση παρέχει πιο πολύ δεδομένα από ότι συνεισφέρει στην λογική του συστήματος.
- **Locality of Attribute Access (LAA):** Είναι ο αριθμός των προσβάσεων στις ιδιότητες μια κλάσης από τις μεθόδους της δια το σύνολο των ιδιοτήτων που προσεγγίστηκαν γενικά από τις μεθόδους της.
- **Foreign Data Provider:** Σχετίζεται με την ύπαρξη ιδιοτήτων ορισμένων ως τύπου άλλης κλάσης οι οποίες χρησιμοποιούνται για την άμεση πρόσβαση σε ξένα δεδομένα.

2.5 Τεχνολογίες Που Χρησιμοποιήθηκαν

Η δημιουργία του συγκεκριμένου Web Application βασίστηκε σε τεχνολογίες και τεχνικές όπως HTML5 , CSS3 , Javascript , JQuery , AJAX, Vis.js, Bootstrap , JSON και REST.

2.5.1 Εμφανισιακό τμήμα

Για την εμφάνιση του Web Application χρησιμοποιήθηκε η Bootstrap ένα εργαλείο ανοιχτού κώδικα όπου προσθέτει δυνατότητες και χαρακτηριστικά στην χρήση HTML και CSS για αποτέλεσμα με καλύτερη απόκριση σε όλες τις συσκευές αλλά και υψηλή καλαισθησία.

2.5.2 Λειτουργικό τμήμα και Visualization

Η κύρια λειτουργικότητα του συστήματος υλοποιήθηκε με την χρήση της Javascript, μιας γλώσσας που αποτελεί τον κορμό για τα Web συστήματα και με ολοένα αυξανόμενο αριθμό βιβλιοθηκών. Συνδυάστηκε ακόμη η JQuery μία βιβλιοθήκη όπου βοηθάει στην μετακίνηση μεταξύ των στοιχείων του DOM (Document Object Model) , στην δημιουργία συμβάντων αλλά και στην υλοποίηση της AJAX (Asynchronous JavaScript and XML). Ακόμη έγινε κατανάλωση υπηρεσιών ιστού REST(Representational State Transfer) οι οποίες είναι απλές, ευκολά διατηρήσιμες και κλιμακώσιμες. Τα δεδομένα τους και η λειτουργικότητα τους θεωρούνται πόροι και είναι προσβάσιμοι μέσω των συνδέσμων

URIs (Uniform Resource Identifiers). Για τον ορισμό σωστών URI χρησιμοποιούμε ουσιαστικά και όχι ρήματα, καθώς δεν θέλουμε να γίνει κάποια ενέργεια και μετά να εμφανιστούν τα δεδομένα μας, είναι σαν να υπάρχουν ήδη. Για την μεταφορά των δεδομένων μεταξύ του browser και του server αλλά και για την αντικατάσταση της XML στην ασύγχρονη επικοινωνία (AJAX) που είδαμε παραπάνω χρησιμοποιήθηκε η μορφή αρχείου JSON. Παραδείγματος χάριν παρακάτω παραθέτετε ένα REST GET αίτημα σε resource based URI μαζί με τα επιστρεφόμενα δεδομένα σε JSON.

GET

<http://se.uom.gr:8080/seagle2/rs/project/versions?purl=https://github.com/jOOQ/jOOR.git>

Επιστρέφει : Status 200 OK (το αίτημα ολοκληρώθηκε επιτυχώς) και την JSON λίστα με όλα τις γενιές για το συγκεκριμένο git έργο.

```
{
  "versions": [
    {
      "commitHashId": "3985f48a2d7dd6615cf78ffee5088ca80698c58a",
      "date": "2012-04-29T00:00:00Z",
      "name": "version-0.9.3",
      "versionDbId": 790
    },
    {
      "commitHashId": "05f15132a3f14b992d0d4a1611d5b5952a0e8d83",
      "date": "2014-06-25T00:00:00Z",
      "name": "version-0.9.4",
      "versionDbId": 791
    },
    {
      "commitHashId": "2743b078b8d0095012d0915aee6ba2ebef036667",
      "date": "2015-03-05T00:00:00Z",
      "name": "version-0.9.5",
      "versionDbId": 792
    },
    {
      "commitHashId": "0c76edf715083143d129846dc77c794d345fbed3",
      "date": "2016-07-06T00:00:00Z",
      "name": "version-0.9.6",
      "versionDbId": 793
    },
    {
      "commitHashId": "4ee88b57d529bb7a4d25b5c8e8b1d757e09eda2a",
      "date": "2017-11-30T00:00:00Z",
      "name": "version-0.9.7",
```

```
"versionDbId": 794
    }
  ]
}
```

Για το **Visualization** του γράφου χρησιμοποιήθηκε η ανοιχτού λογισμικού βιβλιοθήκη Vis.js (<http://visjs.org/>) . Μεσώ της Vis.js δίνεται η δυνατότητα να διαχειριστείς τα δεδομένα σου και να τα αναπαράγεις γραφικά δημιουργώντας δυναμικά και διαδραστικά διαγράμματα (2D και 3D). Αναλυτική παρουσίαση στο Κεφ. 4.1.2 Γραφική αναπαράσταση γράφου

3 Απαιτήσεις Εφαρμογής

3.1 Πρόβλημα

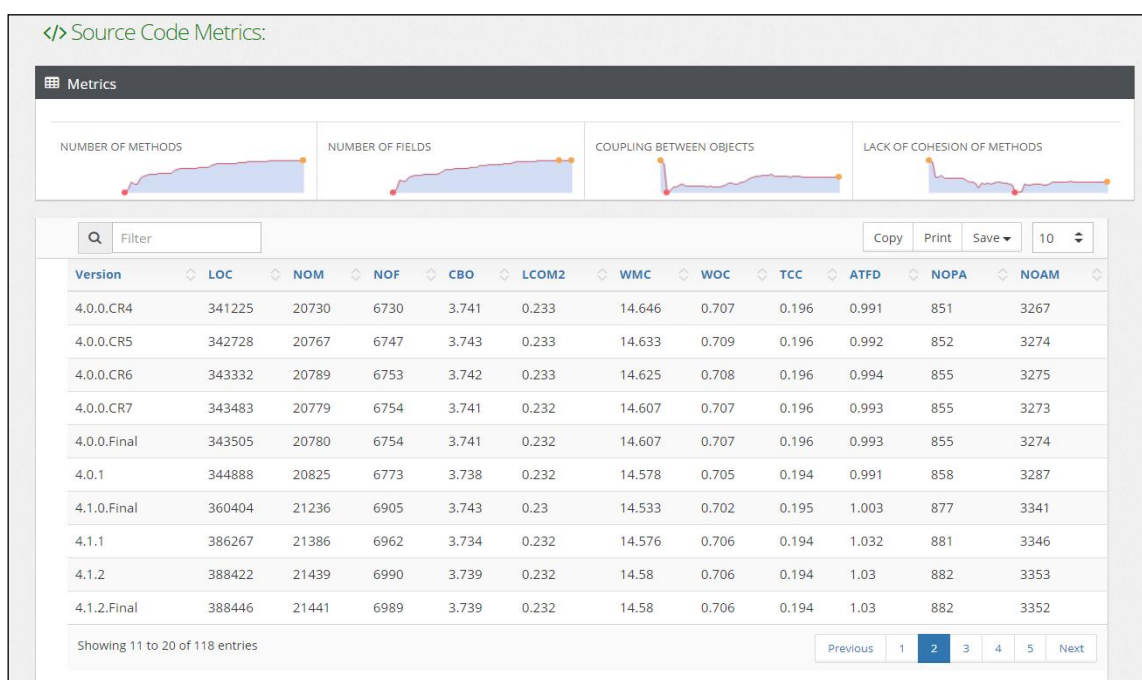
Η σχεδίαση ενός συστήματός λογισμικού είναι μια δύσκολη και χαοτική διαδικασία. Χαοτική καθώς υπάρχουν παρά πολλές μεταβλητές που πρέπει να ληφθούν υπόψη, κάποιες από αυτές είναι ο καθορισμός των λειτουργικών απαιτήσεων, των τεχνικών απαιτήσεων, την επιλογή της κατάλληλης αρχιτεκτονικής και προτύπων για την επικοινωνία των διαφόρων τμημάτων. Ακόμη στο μυαλό του σχεδιαστή θα πρέπει να βρίσκονται και οι μελλοντικές αλλαγές που θα χρειαστούν τόσο για την πιθανή επέκταση του έργου αλλά και σιγουρά για την συντήρηση του. Συμφώνα με τον Erlikh [11] το κύριο κόστος ενός έργου αλλά και συνήθως το πιο δύσκολο κομμάτι του είναι η μακροπρόθεσμή συντήρηση του. Οι αλλαγές για να γίνονται ευκολά απαιτούν από τους προγραμματιστές να έχουν υψηλή κατανόησή για το σύνολο του συστήματος. Σε αντίθεση με την συγγραφή καινούργιου κώδικα κάτι τέτοιο είναι πραγματικά πολύ δύσκολο όσο το έργο μεγαλώνει ή νέοι προγραμματιστές έρχονται σε επαφή με παλιό κώδικα (legacy code) . Σύμφωνα με τους Fjeldstad και Hamlen [12] γνωρίζουμε από έρευνες ότι οι προγραμματιστές για να συντηρήσουν το λογισμικό περνάνε το 50% του χρόνου τους για να το κατανοήσουν.

Μεταξύ κατανόησης και κακής αντικειμενοστρεφούς σχεδίασης υπάρχει άμεση σχέση. Για παράδειγμα όταν ένα σύστημα έχει μεγάλες και πολύπλοκες κλάσεις , πολλές αλληλεξαρτήσεις , δεν υπάρχει ξεκάθαρος διαχωρισμός αρμοδιοτήτων και γενικά υπάρχει ελλείψει αφαιρετικής σκέψης τότε η κατανόηση ενός τέτοιου συστήματός απαιτεί εξαιρετική προσπάθεια. Από την άλλη μεριά η έλλειψη κατανόησης του προγράμματος από έναν προγραμματιστή είναι πιθανόν να οδηγήσει σε χαμηλή παραγωγικότητα ή σε υλοποιήσεις που δεν ακολουθούν την αρχική σχεδίασή και δημιουργούν πρόβλημα ή θα δημιουργήσουν μελλοντικά.

Συνεπώς ο κύριος στόχος μας είναι να διευκολύνουμε την κατανόηση μεγάλων συστημάτων λογισμικού αλλά κυρίως τον εντοπισμό προβλημάτων κακής αντικειμενοστρεφούς σχεδίασης.

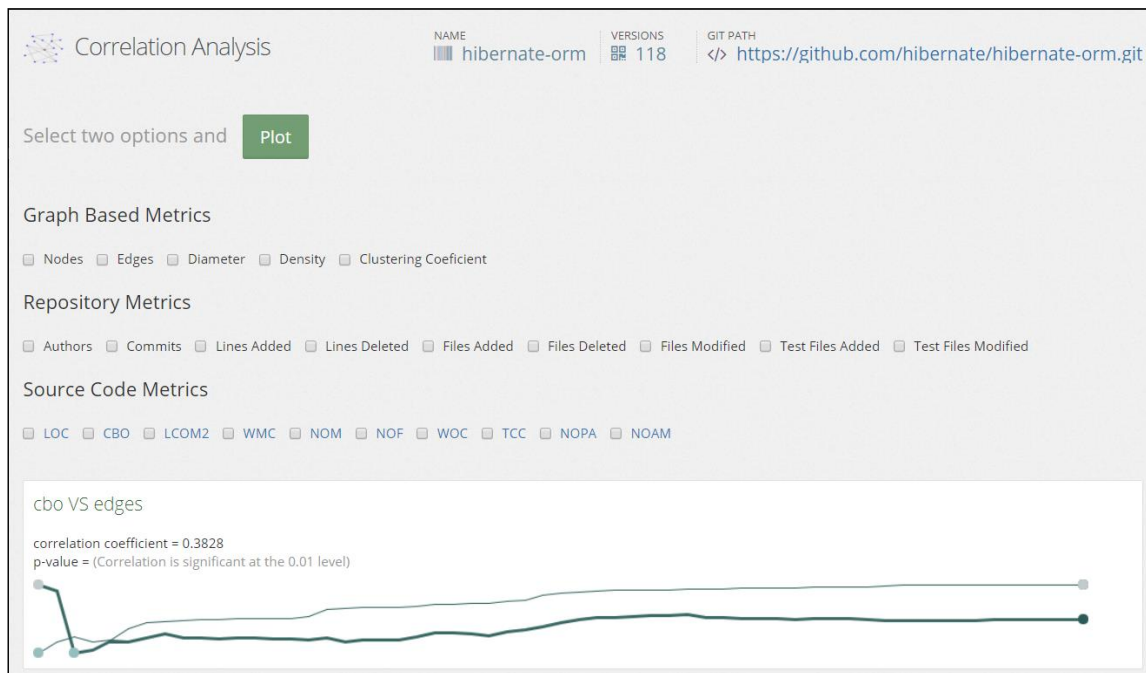
3.2 Που βασίστηκε η εφαρμογή

Η προσπάθεια μας αυτή βασίστηκε στην ηλεκτρονική πλατφόρμα ανάλυσης λογισμικού SEAGle (<http://se.uom.gr/seagle/>) το οποίο αναπτύχθηκε στο εργαστήριο τεχνολογίας λογισμικού. Το Seagle αρχικά συλλέγει τα δεδομένα για ένα δοθέν αποθετήριο git και στην συνέχεια υπολογίζει μετρικές γράφου, πηγαίου κώδικα, προβλήματα σχεδίασης (Code smells) αλλά και μετρικές σχετικά με το git repository και την διαδικασία ανάπτυξης. Ακόμη επειδή το κύριο μέλημά του είναι η παρουσίαση της εξελικτικής πορείας ενός project , υπάρχουν διαγράμματα και πίνακες με τις τιμές για κάθε γενιά. Στην Εικόνα 1 γίνεται παρουσίαση των αποτελεσμάτων για τις μετρικές πηγαίου κώδικα συνολικά για κάθε γενιά του έργου Hibernate (<https://github.com/hibernate/hibernate-orm.git>).



Εικόνα 1 : Εμφάνιση Δεδομένων SEAGle

Το SEAGle παρέχει ακόμη την δυνατότητα για ανάλυση της συσχέτισης δύο οποιαδήποτε μετρικών κατά την πορεία των γενεών. Στην Εικόνα 2 θέλουμε να δούμε την συσχέτιση που υπάρχει ανάμεσα της σύζευξης μεταξύ αντικειμένων (CBO metric) και πλήθους ακμών.



Εικόνα 2 : Correlation Analysis SEagle

3.3 Επέκταση

Η επέκταση στην οποία στοχεύουμε είναι να δώσουμε στους προγραμματιστές την δυνατότητα να εμβαθύνουν στην κάθε γενιά μέσω visualization γράφου.

Πιο συγκεκριμένα η πρώτη κίνηση μας είναι να αναπαραστήσουμε γραφικά το project με την μορφή γράφου. Ο γράφος θα αποτελείται από κόμβους για κάθε κλάση και ακμές για τις σχέσεις μεταξύ τους, έτσι θα μπορούμε να δούμε καλύτερα την αρχιτεκτονική του συστήματος και τις επικοινωνίες μεταξύ των κλάσεων. Με τον τρόπο αυτό επιτυγχάνουμε ευκολότερη και γρηγορότερη κατανόηση μεγάλων project ή παλιών συστημάτων λογισμικού (legacy code).

Η δεύτερη κίνηση μας είναι να συνδυάσουμε της μετρικές για κάθε κλάση με την δυναμική γραφική αναπαράσταση του έργου για κάθε γενιά. Ο χρήστης θα μπορεί να αλλάξει την κλίμακα του γράφου ή και να τον φιλτράρει με βάση τις τιμές των κόμβων-κλάσεων του σε μία επιλεγμένη μετρική. Το αποτέλεσμα αυτού θα είναι να παίρνουμε άμεσα πληροφορίες για το τι συμβαίνει μέσα σε κάθε κλάση αλλά και μεταξύ τους, έτσι ο εντοπισμός προβλημάτων κακής αντικειμενοστρεφούς σχεδίασης ή άλλων ανωμαλιών σε μεγάλα project γίνεται ευκολότερος.

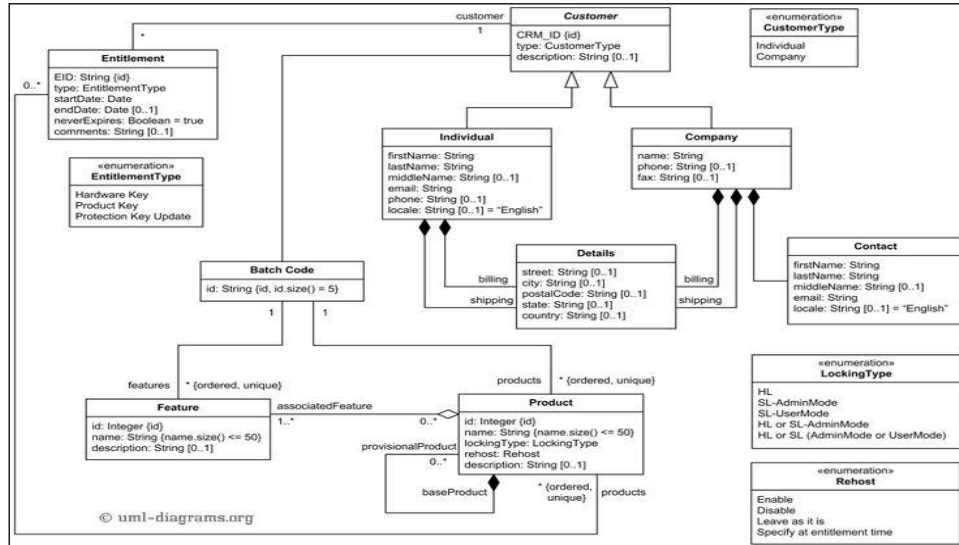
3.4 Σύγκριση Παρόμοιων Εργαλείων

3.4.1 Γενικά

Για την γραφική αναπαράσταση έργων που βασίζονται σε αντικειμενοστρεφή σχεδίαση υπάρχει στην αγορά πληθώρα εργαλείων όπου μπορούν να χωριστούν σε δυο κατηγορίες.

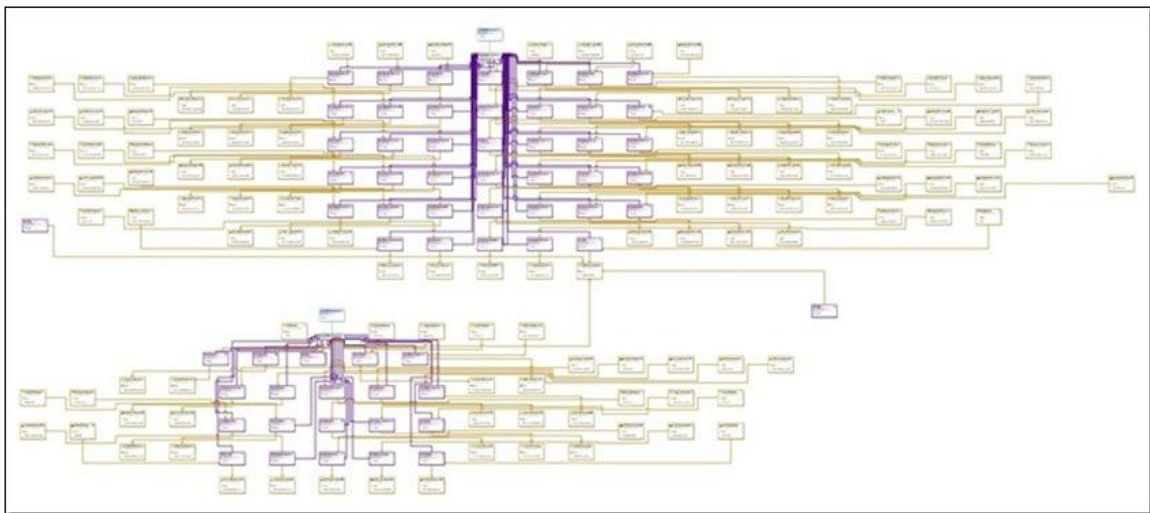
Στην πρώτη κατηγορία είναι τα εργαλεία όπου αναπαριστούν το σύστημα σε μορφή γράφου ή σε κάποια άλλη μορφή. Τα αρνητικά αυτής της κατηγορίας εργαλείων είναι ότι πολλές από τις διαφορετικές μορφές που επιλέγονται για visualization είναι δυσνόητες και απαιτούν επιπλέον προσπάθεια για την κατανόηση της αρχιτεκτονικής του έργου, όπου συνήθως είναι αδύνατη να επιτευχθεί από την μελέτη μόνο του σχήματος. Αλλά ακόμη και στις περιπτώσεις όπου επιλέγεται ο γράφος για το visualization και είναι δυνατή η κατανόηση των σχέσεων μεταξύ των κλάσεων, δεν μπορεί να εξαχθεί από το σχήμα κάποια επιπλέον πληροφορία καθώς τις περισσότερες φορές δεν υπάρχουν επιπλέον ρυθμίσεις για ουσιαστική διαχείριση και μετατροπή του γράφου.

Η δεύτερη και πιο συχνή κατηγορία είναι αυτή των εργαλείων που βασίζονται στην UML (Unified Modeling Language) . Η UML αποτελεί μία ευρύτερα διαδεδομένη γλώσσα μοντελοποίησης λογισμικού όπου χρησιμοποιείται κυρίως κατά την φάση της σχεδίασης. Υπάρχουν δύο κατηγορίες τύπων διαγραμμάτων για την UML, τα διαγράμματα δομής αλλά και τα διαγράμματα συμπεριφοράς. Το πιο σύννηθες διάγραμμα είναι το διάγραμμα κλάσεων όπου ανήκει στην πρώτη κατηγορία (Εικόνα 3).



Εικόνα 3 : UML class diagram

Στην παραπάνω εικόνα γίνεται αναπαράστασή 13 κλάσεων με πληροφορίες για της ιδιότητες και της μεθόδους που υπάρχουν για κάθε κλάση αλλά και πληροφορίες για της σχέσεις μεταξύ τους. Όλες αυτές οι πληροφορίες είναι σημαντικές αλλά χάνετε η αξία της απεικονίσεις τους όταν θέλουμε να δούμε το σύνολο ενός συστήματος όπως γίνετε σαφές από την Εικόνα 4.



Εικόνα 4 : Μεγάλης κλίμακας UML διάγραμμα

Με την Εικόνα 4 είναι δύσκολο να εντοπιστούν προβλήματα κακής σχεδίασης και ανωμαλίες στο σύστημα. Η εμφάνιση των πληροφοριών που μας προσφέρει η UML για

το τι συμβαίνει μέσα σε κάθε κλάση μπορεί να αντικατασταθεί από το συνδυασμό γράφου και μετρικές. Αυτό συμβαίνει γιατί οι μετρικές έχουν την δυνατότητα να μας δίνουν μια αναφορά για όλα αυτά που συμβαίνουν σε κάθε κλάση χωρίς να χρειάζεται να διαβάσουμε ούτε καν το outline της. Για παράδειγμα δεν χρειάζεται να διαβάσουμε την λίστα με της ιδιότητες και να δούμε ότι υπάρχουν πολλές από αυτές που είναι public , αυτό το βλέπουμε πολύ ευκολά αν ρυθμίσουμε το scaling ή φιλτράρουμε τον γράφο με βάση την μετρική NOPA (Number of public attributes).

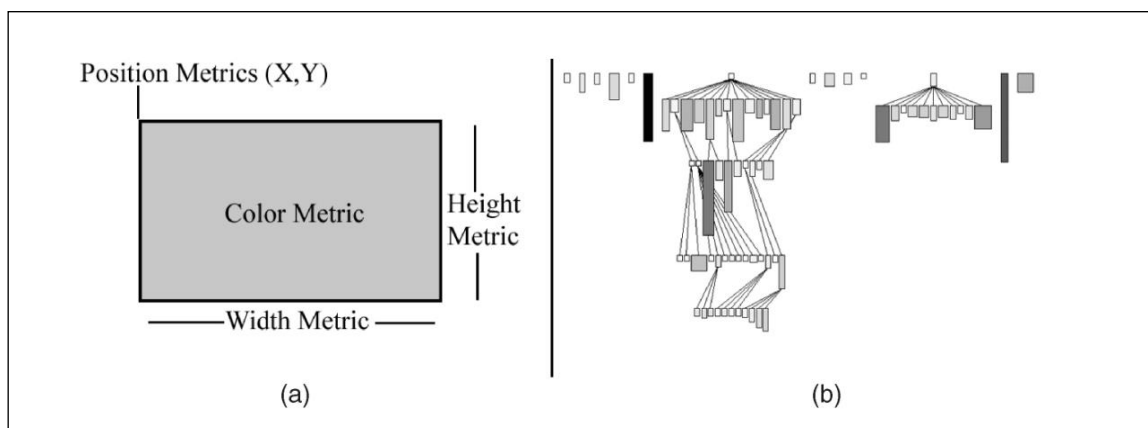
3.4.2 Ερευνητικά Εργαλεία που Ξεχωρίζουν

Σε αντίθεση με τα περισσότερα από τα παραπάνω εργαλεία υπάρχουν πολλά ερευνητικά έργα που προσφέρουν ιδιαίτερους τρόπους απεικόνισής του συστήματος. Παρακάτω γίνεται μια αναφορά σε κάποια από αυτά.

Ένα από τα πρώτα έργα γραφικής απεικόνισής λογισμικού είναι το Rigi του Muller και Klashinsky το 1988 [13]. Το Rigi ήταν ένα δίκτυο-γράφος όπου κύριο σκοπό είχε να δείξει της αλληλεξάρτησης μεταξύ των θεμέλιων λίθων ενός προγράμματος όπως ήταν τα διάφορα υποσυστήματα, implementations και συλλογές δεδομένων.

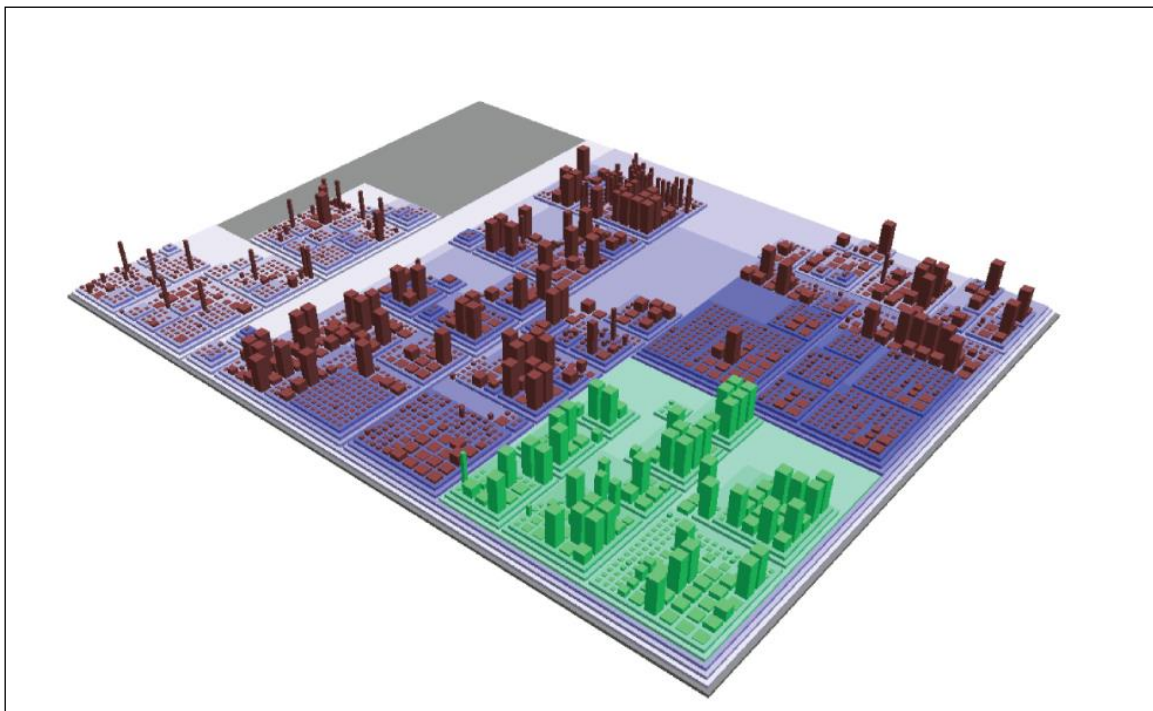
Ο Lanza το 2002 [14] και στην συνέχεια το 2003 [15] με προσθήκες κυρίως για να καλύψει την ανάγκη για επεκτασιμότητα ενός έργου λογισμικού, συνδύασε γράφο με ιεραρχική διάταξη (δέντρο) και τετραγωνισμένους κόμβους για την αναπαράσταση των μετρικών. Πιο συγκεκριμένα όπως βλέπουμε στην Εικόνα 5 μπορεί να γίνει αναπαράσταση 5 διαφορετικών μετρικών ταυτόχρονα με βάση :

- Το ύψος του κόμβου.
- Το πλάτος του.
- Το χρώμα του.
- Την θέση του στον άξονα X.
- Την θέση του στον άξονα Y.



Εικόνα 5 : Lanza Visualization

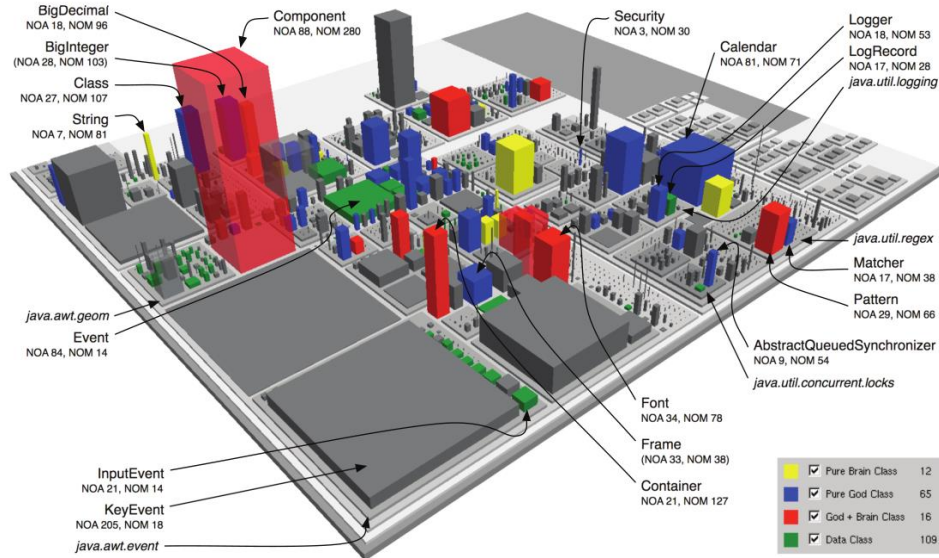
Η γραφική απεικόνιση συστημάτων δεν περιορίζεται μόνο στις δύο διαστάσεις. Ένα εντυπωσιακό 3D έργο είναι αυτό των Wettel και Lanza το 2007 [16] όπου αναπαριστούν ένα αντικειμενοστραφή σύστημα λογισμικού ως μία πόλη. Ο χρήστης έχει την δυνατότητα να περιπλανηθεί σε αυτή δίνοντας του έτσι μια καλύτερη αίσθηση του χώρου και συνεπώς του συστήματος. Το κάθε κτίριο αποτελεί μια κλάση και το κάθε οικοδομικό τετράγωνο και περιοχή ένα πακέτο. Ο ύψος των κτιρίων είναι ανάλογο του πλήθους των μεθόδων της κλάσης που αναπαριστά, ενώ το πλάτος και το μήκος του βασίζεται στο πλήθος των ιδιοτήτων. Ο χρωματισμός των οικοδομικών τετραγώνων είναι ανάλογος του βαθμού φωλιάσματος του πακέτου. Ακόμη οι βιβλιοθήκες που χρησιμοποιεί το έργο σχεδιάζονται γραφικά ως τα προάστια της πόλης *Εικόνα 6*.



Εικόνα 6 : Επιλογή περιοχής στο έργο ArgoUML

Η αδυναμία της παραπάνω προσέγγισης ήταν ότι δεν υπήρχε δυνατότητα για αναπαράσταση και συνδυασμό πολλών μετρικών. Το πρόβλημα αυτό το λύνουν το 2008 [17] εισάγοντας έναν τρόπον χρωματισμού των κτιρίων όπου παρομοιάζεται με χάρτη ασθενειών. Πιο συγκεκριμένα όπως μπορούμε να δούμε το τύπο ασθενειών που πλήττουν τον κόσμο αλλά και τον τρόπο που είναι διανεμημένες σε αυτόν, έτσι μπορούμε να δούμε

και ποια προβλήματα κακής σχεδίασης (Code Smells) πλήττουν το σύστημα μας (Εικόνα 7).



Εικόνα 7 : Code smells και City Visualization

Μια διαφορετική οπτική στον εντοπισμό προβλημάτων παραγωγικότητας σε γραφικές αναπαραστάσεις συστημάτων ως πόλεις δίνεται από τους Panas , Berrigan και Grundy [18] . Στο έργο τους τα κομμάτια του συστήματος οπού χρησιμοποιούνται πολύ συχνά αναπαριστώνται ως φλεγόμενα κτήρια. Με τον τρόπο αυτό εντοπίζονται ευκολά τα σημεία οπού πρέπει να προσέξουμε προκειμένου να αυξήσουμε την αξιοπιστία και της επιδόσεις του συστήματος μας. Ακόμη για την μείωση του κόστους συντηρήσεις τα συστατικά μέρη όπου υποπίπτουν σε συχνές αλλαγές εμφανίζονται ως κτήρια με κεραυνούς (Εικόνα 8).



Εικόνα 8 : Πόλη με business προβλήματα

4 Σχεδίαση και υλοποίηση εφαρμογής

4.1 Τεχνικό κομμάτι εργασίας

Για την δημιουργία της διαδικτυακής εφαρμογής στην οποία γίνεται η επιλογή του αναλυμένου έργου, της γενιάς που μας ενδιαφέρει και στην συνέχεια visualization και επεξεργασία του γράφου χρησιμοποιήθηκαν κυρίως HTML5, CSS3, Javascript , AJAX , Json, Rest, Bootstrap και για τον σχεδιασμό του γράφου η open-source βιβλιοθήκη Vis.js (<http://visjs.org/>) .

Θα μπορούσαμε να χωρίσουμε την διαδικτυακή εφαρμογή σε τρία κομμάτια.

4.1.1 Επιλογή Έργου και Γενιάς

Το πρώτο κομμάτι είναι η αρχική σελίδα στην οποία γίνεται η αναζήτηση και η επιλογή του έργου που έχει αναλυθεί από την πλατφόρμα του Seagle και της γενιάς που θέλουμε να δούμε στον γράφο.

Αρχικά υπάρχει ένα select menu όπου εμπεριέχει όλα τα αναλυμένα έργα. Όταν ο χρήστης επισκέπτεται την εφαρμογή λαμβάνει χώρα ένα GET request στο REST URI <http://se.uom.gr:8080/seagle2/rs/project/> με την χρήση AJAX. Το παραπάνω URI επιστρέφει σε μορφή JSON την λίστα των έργων όπου για κάθε έργο δίνει το όνομα του, το timestamp της χρονικής στιγμής που έγινε η ανάλυση, το url του στο github όπου βρίσκεται ο πηγαίος κώδικας και τον αριθμό των γενεών που περιέχει. Στην συνέχεια προσαρτιέται στο παραπάνω select menu το όνομα και το url για κάθε έργο. Επίσης δίνεται η δυνατότητα στον χρήστη να περιορίσει την λίστα των αποτελεσμάτων πληκτρολογώντας τα αρχικά του έργου. Αυτή η λειτουργία γίνεται με την βοήθεια της JQuery βιβλιοθήκης για Bootstrap την Select2 (<https://github.com/select2/select2>).

Μόλις γίνει η επιλογή του project που μας ενδιαφέρει τότε γίνεται ένα δεύτερο ασύγχρονο GET request αυτή την φορά στο <http://se.uom.gr:8080/seagle2/rs/project/versions?purl=projectName> , όπου ως τιμή του projectName έχουμε το όνομα του project που διάλεξε παραπάνω ο χρήστης. Από το JSON response χρησιμοποιούμε τα ονόματα των versions για να γεμίσουμε το dropDown μενού που σχετίζεται μαζί τους. Επιλέγοντας το version που ενδιαφέρει τον χρήστη συλλέγουμε το επιλεγμένο version ID και εμφανίζουμε το κουμπί “Show Version’s

Graph” όπου όταν πατηθεί μας μεταφέρει στην δεύτερη σελίδα μαζί με 2 παραμέτρους , το purl οπού ισούται με το git url του project και το versionID όπου περιέχει το μοναδικό αναγνωριστικό του version.

4.1.2 Γραφική αναπαράσταση γράφου

Το δεύτερο κομμάτι του site είναι η σελίδα οπού περιέχει την γραφική αναπαράσταση του γράφου και το πλαϊνό μενού με όλες τις επιλογές για την επεξεργασία του. Για το visualization του γράφου χρησιμοποιείτε javascript , jQuery και η βιβλιοθήκη Vis.js (<http://visjs.org/>).

Η βιβλιοθήκη **Vis.js** μια από τις δυνατότητες που προσφέρει όσο αναφορά την διαχείριση των δεδομένων είναι η σύνδεση μεταξύ του γράφου-δίκτυο(network) και των δυναμικών συνόλων δεδομένων (DataSet) οπού επιτρέπουν την επεξεργασία των δεδομένων. Το DataSet είναι μία ευέλικτη δομή κλειδιού / τιμής για την διαχείριση JSON δεδομένων. Το κύριο πλεονέκτημα των DataSet είναι ότι από την στιγμή που συνδεθεί με το γραφό-δίκτυο ότι αλλαγή συμβεί στα δεδομένα όπως εισαγωγή, ενημέρωση, διαγραφή , φιλτράρισμα μεταφέρεται άμεσα και στην γραφική αναπαράσταση. Για τον λόγο αυτό δημιουργούμε δυο DataSets, ένα για τους κόμβους και έναν για τις ακμές και τα περνάμε ως ιδιότητες σε ένα Javascript αντικείμενο data οπού παρέχει τα δεδομένα στην Vis διάταξη.

Ένα δεύτερο σημαντικό αντικείμενο για το Vis είναι το αντικείμενο options όπου εμπεριέχει όλες της ρυθμίσεις που θέλουμε να αλλάξουμε για το visualization του γράφου. Κάποιες από αυτές είναι οι εξής:

- Ακμές (Edges) : Για την δημιουργία και διαγραφή των ακμών και εδώ δηλώνονται οι global ρυθμίσεις και τα στυλ των ακμών. Παραδείγματος χάριν:
 - Επιλέγουμε αν θα είναι κατευθυνόμενες ή όχι.
 - Το χρώμα τους.
 - Την ομαλότητα κατά την μετακίνηση τους
 - Την κυρτότητα τους.
- Κόμβοι (Nodes) : Για την δημιουργία και διαγραφή των κόμβων, επίσης δηλώνονται οι

global ρυθμίσεις και τα στυλ των κόμβων. Παραδείγματος χάριν:

- Το αρχικό τους σχήμα , στην περίπτωση μας είναι η τελεία.
- Το χρώμα της γραμματοσειράς της ταμπέλας τους.
- Ορίζουμε την συνάρτηση για το scaling (Εικόνα 9). Τα αρχικά min και max δηλώνουν το πόσο μικρός ή μεγάλος μπορεί να γίνει ένας κόμβος. Ακόμη έχουμε απενεργοποίηση το scaling για την ταμπέλα του κόμβου ώστε να μένει σταθερή ανεξαρτήτως του μεγέθους του κόμβου.
- Η συνάρτηση που ορίζουμε για το scaling έχει 4 παραμέτρους. Την ελάχιστη και μέγιστη τιμή του πεδίου value για κάθε κόμβο, το συνολικό άθροισμα των τιμών του value και το πεδίο value.

```
scaling: {  
    min: 10,  
    max: 70,  
    label: {  
        enabled: false,  
        min: 14,  
        max: 30,  
        maxVisible: 30,  
        drawThreshold: 5  
    },  
    customScalingFunction: function (min,max,total,value) {  
        if (max === min) {  
            return 0.5;  
        }  
        else {  
            let scale = 1 / (max - min);  
            return Math.max(0,(value - min)*scale);  
        }  
    }  
}
```

Εικόνα 9: Ρυθμίσεις Scaling

- Φυσική (Physics) : Ορίζει την φυσική προσομοίωση του γράφου. Παραδείγματος χάριν:
 - Μέγιστη ταχύτητα (Velocity): Η μέγιστη ταχύτητα του κόμβου κατά την δημιουργία του.
 - Ελάχιστη ταχύτητα : Μόλις η ταχύτητα κάθε κόμβου φτάσει στην συγκεκριμένη τιμή τότε θεωρούμε ότι η σταθεροποίηση (stabilization) έχει

πραγματοποιηθεί.

- **Barnes-Hut** : Είναι ένα quadtree βαρυτικό μοντέλο οπού επιλέχθηκε για την περίπτωση μας καθώς είναι η γρηγορότερη λύση για μη-ιεραρχικές διατάξεις. Μπορούμε να ορίσουμε την τιμή της βαρύτητας χρησιμοποιώντας αρνητικές τιμές καθώς θέλουμε να υπάρχει αποστροφή μεταξύ των κόμβων. Ακόμη ορίζουμε την κεντρική βαρύτητα η οποία ελκύει ολόκληρο τον γράφο πίσω στο κέντρο. Για τις ακμές ορίζουμε το μήκος τους και το πόσο ισχυρά είναι στις μετακινήσεις.
- Αλληλεπίδραση (Interaction) : Χρησιμοποιείτε για όλες τις αλληλεπίδρασης του χρήστη με τον γράφο. Παραδείγματος χάριν:
 - Επιλέγουμε ότι κατά την επιλογή ενός κόμβου θα τονίζονται οι ακμές του.
- Διάταξη (Layout) : Ρυθμίζει την αρχική θέση των κόμβων, το animation και το ζουμ. Παραδείγματος χάριν
 - Έχουμε επιλέξει να γίνετε ζουμ μετά από ένα συγκεκριμένο στάδιο της αρχικής τοποθέτησης των κόμβων ώστε να χωρέσουν όλοι στην οθόνη του χρήστη.

Αφού έχει γίνει ο ορισμός των βασικών ρυθμίσεων κάνουμε ένα GET request στο URI <http://se.uom.gr:8080/seagle2/rs/project/graph/forVersion/> + η τιμή του versionID. Το παραπάνω request μας επιστρέφει ένα JSON αρχείο της μορφής

```
“edge” : [ { “from” : initial full class name , “to” : destination’s full class name } , ... ] ,  
“nodes” : [ { “id” : full class name , “label” : full class name , “value”: null } ]
```

Σε πρώτη φάση προσθέτουμε και υπολογίζουμε δυο επιπλέον πεδία για κάθε κόμβο το “IN” και “OUT” , οπού το IN αναφέρεται στο in-degree του συγκεκριμένου κόμβου δηλαδή το πλήθος των ακμών που καταλήγουν σε αυτόν και το OUT είναι το out-degree δηλαδή πόσες ακμές ξεκινάνε από τον συγκεκριμένο κόμβο.

Για τον σχεδιασμό των κλάσεων στην περίπτωση που έχουν τακτοποιηθεί ως προβλήματα σχεδίασης (Code smells) χρησιμοποιούμε AJAX για να κάνουμε ένα GET request στο Rest URI <http://se.uom.gr:8080/seagle2/rs/project/smells/forVersion?versionID> = η τιμή του επιλεγμένου version. Με βάση τις πληροφορίες που συλλέγουμε από το παραπάνω JSON αρχείο αλλάζουμε το σχήμα του κόμβου από κουκίδα (dot) που είναι αρχικά σε αστέρι, ρόμβο ή τετράγωνο αναλόγως το είδος του code smell.

Ακόμη εισάγουμε ένα πεδίο “Group” για κάθε κόμβο με τιμή το πακέτο όπου βρίσκεται η αντίστοιχη κλάση ώστε να εφαρμοστεί το ίδιο χρώμα σε κάθε ομάδα.

Μόλις γίνουν οι παραπάνω αρχικές εργασίες εισάγουμε τους επεξεργασμένους κόμβους και ακμές στα αντίστοιχα DataSets και στην συνέχεια αρχικοποιούμε το vis.network με παραμέτρους ένα div container , τα DataSets και το αντικείμενο options.

4.1.3 Μπάρα ρυθμίσεων

Η πλαϊνή μπάρα αποτελείται από την επιλογή της μετρικής για το scaling του γράφου, το φιλτράρισμα των κόμβων αναλόγως των αντίστοιχων τιμών τους στην επιλεγμένη μετρική και τέλος την επιλογή της συμπεριφοράς του γράφου κατά την επιλογή ενός κόμβου.

Όταν ο χρήστης διαλέγει μια μετρική από το πρώτο select menu γίνεται ένα Rest GET Request στο [/metric/single/values/project/forVersion?purl](#) = διεύθυνση project & metricMnemonic = μνημονικό επιλεγμένης μετρικής & versionID = αναγνωριστικό version.

Στην συνέχεια ανατίθεται στο πεδίο value του κάθε κόμβου η ανάλογη τιμή του στην συγκεκριμένη μετρική και γίνεται ενημέρωση του Vis DataSet που αφορά τους κομβούς.

Για το φιλτράρισμα των κόμβων με βάση την μετρική , το σύμβολο σύγκρισης και την τιμή που έχει διαλέξει ο χρήστης ορίζουμε μια συνάρτηση όπου ενημερώνει το χρώμα

του κάθε κόμβου ανάλογα την τιμή του. Χρησιμοποιούμε το πράσινο για να δηλώσουμε ότι ικανοποιεί το παραπάνω κριτήριο και γκρι για όταν δεν το ικανοποιεί. Με την συνάρτηση που ορίσαμε και σε συνδυασμό με την επιλογή filter της Vis για τα DataSets οπού είναι πανομοιότυπη με την filter της Javascript ενημερώνουμε τον γράφο με τους νέους τροποποιημένους κόμβους.

Κατά την επιλογή ενός κόμβου έχει οριστεί ένα συμβάν οπού ελέγχει ποιο radio button έχει διαλέξει ο χρήστης και ανάλογα φιλτράρει τα DataSets για το ποιες ακμές και κόμβοι θα εμφανιστούν.

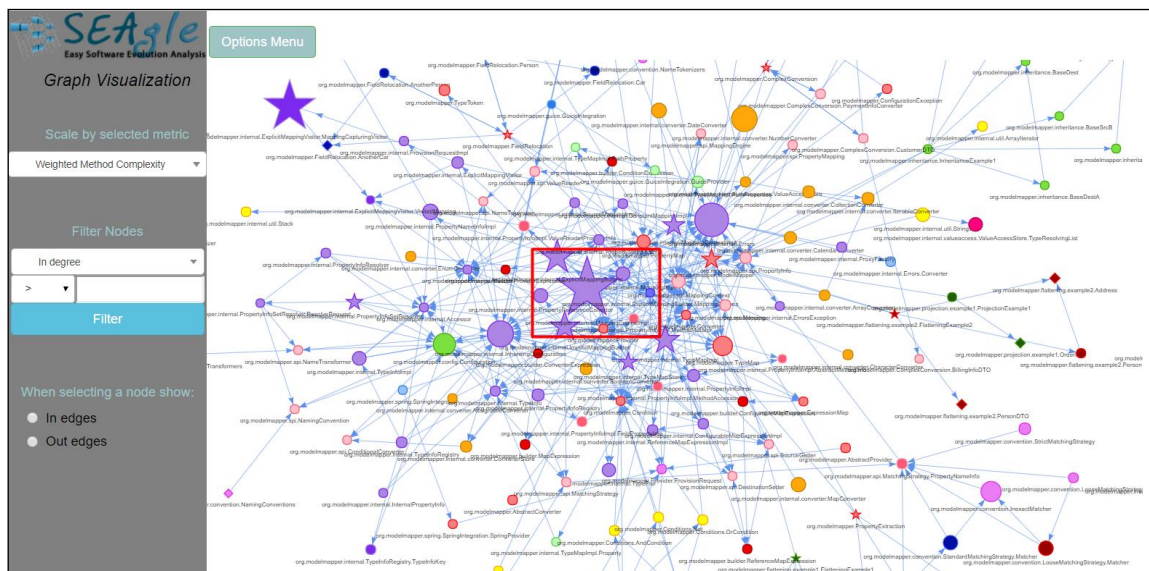
5 Παρουσίαση της εφαρμογής

5.1 Εξαγωγή Πληροφοριών μέσω Visualization

Σε αυτό το σημείο της εργασίας θα δούμε την διαδικασία εξέτασης της γραφικής αναπαράστασης των γράφων , πώς δηλαδή μπορούμε να εξάγουμε πληροφορίες για την καλύτερη κατανόηση ενός συστήματος λογισμικού αλλά και για την εύρεση προβληματικών σημείων στον κώδικα και γενικότερα στην αντικειμενοστρεφή σχεδίαση.

5.1.1 Εντοπισμός God Class

Οι God classes όπως είδαμε και στο [Κεφ 2.2.2](#) αποτελούν κλάσεις με υψηλή πολυπλοκότητα συνεπώς μία καλή αρχή για τον εντοπισμό τους είναι να κάνουμε scale τον γράφο με βάση την μετρική WMC (Weight Method Complexity).

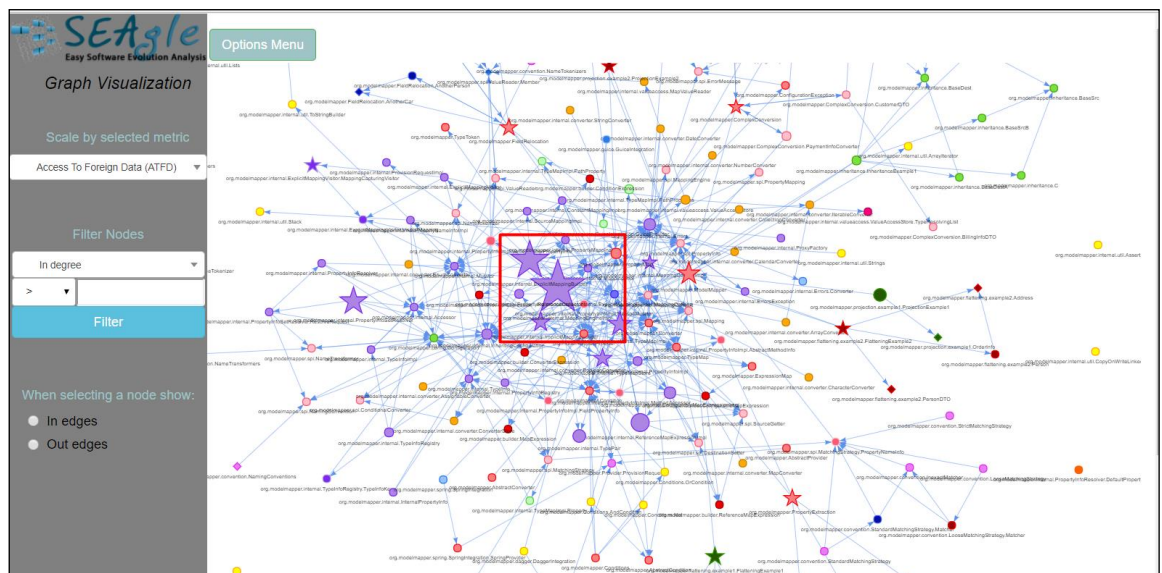


Εικόνα 10 : Τονισμός Πολύπλοκων Κλάσεων

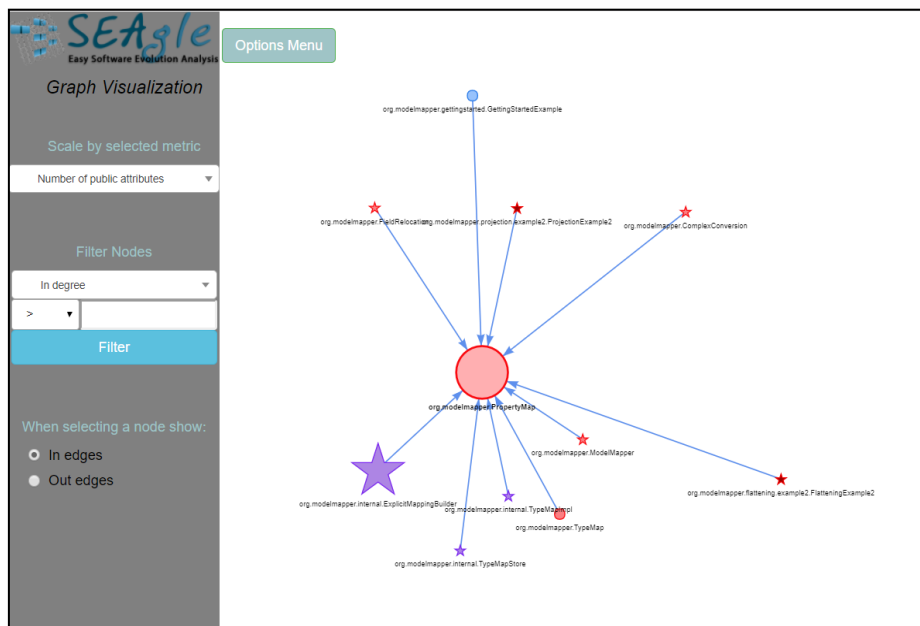
Η κεντρική κλάση του επιλεγμένου σημείου με κόκκινο είναι η MappingEngineImpl του πακέτου org.modelmapper.internal από modelmapper γενιά 1.1.1

Από την παραπάνω εικόνα παρατηρούμε ότι υπάρχουν και κόμβοι με υψηλές τιμές WMC που όμως δεν έχουν χαρακτηριστεί από το σύστημα ως God Classes, αυτό συνήθως σημαίνει ότι δεν ικανοποιούν ακόμα όλες τις προδιαγραφές για να χαρακτηριστούν ως το συγκεκριμένο code smell και χρειάζονται επιπλέον διερεύνηση .

Κύριο χαρακτηριστικό τους είναι ακόμη η τάση τους να χρησιμοποιούν τις γειτονικές τους κλάσεις μόνο για την απόκτηση δεδομένων και συνήθως η πρόσβαση σε αυτά γίνεται άμεσα παραβιάζοντας την Αρχή της ενσωμάτωσης ([Κεφ 2.1](#)). Για τον λόγο αυτό βλέπουμε στην (Εικόνα 11) ότι κλάση MappingEngineImpl εμφανίζει και υψηλή τιμή ATFD (Access to Foreign Data) αλλά και η (Εικόνα 12) μας δείχνει ότι συνήθως κλάσεις με υψηλές τιμές NOPA (Number of Public Attributes) χρησιμοποιούνται ως επι το πλείστον από God Classes.

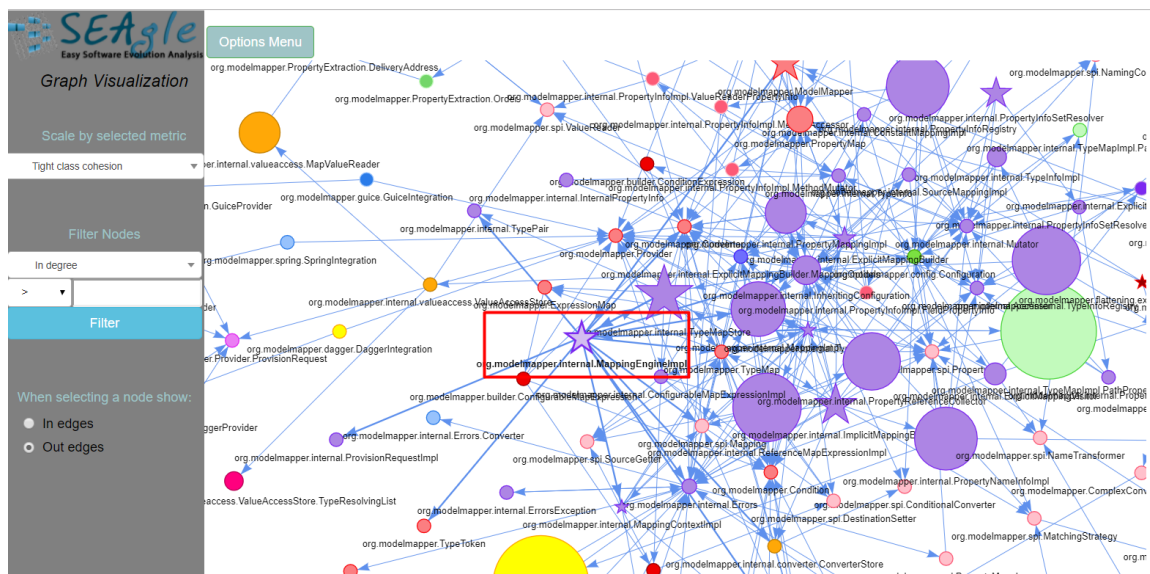


Εικόνα 11 : Τονισμός κλάσεων με υψηλές τιμές ATFD



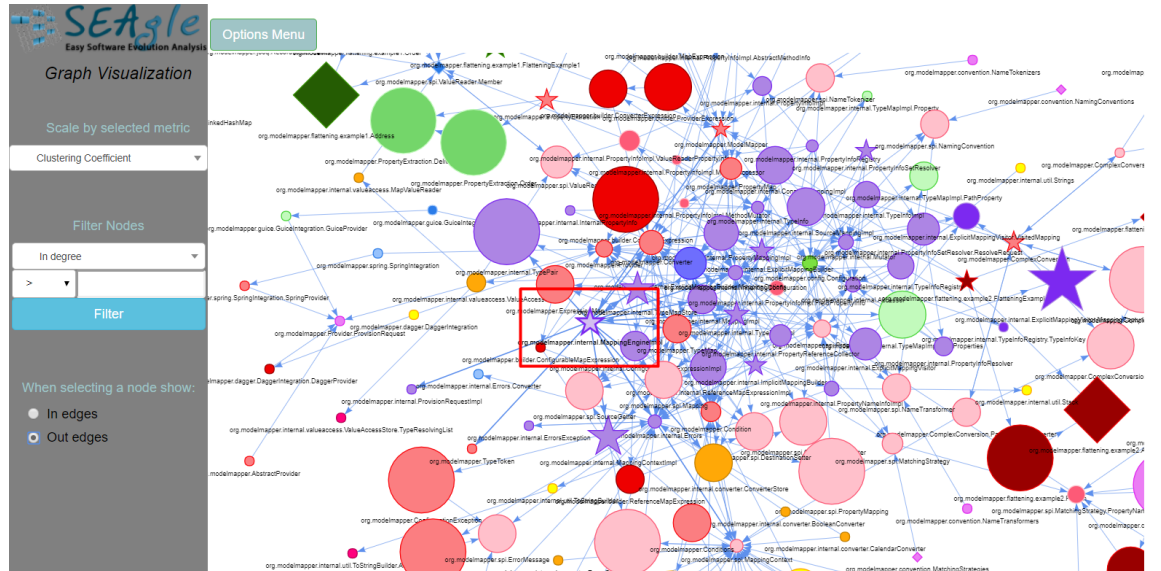
Εικόνα 12 Σχέση μεταξύ NOPA και God Classes.

Ακόμη οι God Classes δεν τηρούν την Αρχή της μοναδικής αρμοδιότητας κάτι που γίνεται ευκολά αντιληπτό από την υψηλή τιμή Out Degree αλλά και αντίστροφα από την χαμηλή τιμή συνεκτικότητας (Εικόνα 13).



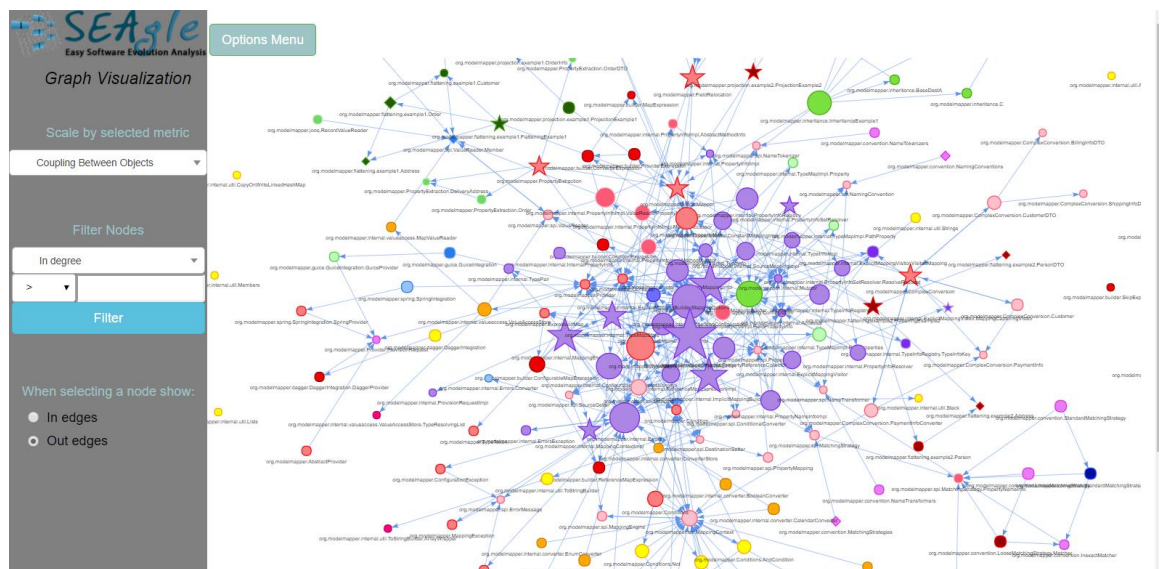
Εικόνα 13 : Συνεκτικότητα

Στην Εικόνα 14 μπορούμε να παρατηρήσουμε ότι η παραπάνω κλάση αλλά και οι περισσότερες God Classes έχουν μικρή τιμή Clustering Coefficient κάτι που είναι λογικό καθώς έχοντας πολλαπλές αρμοδιότητες μια κλάση είναι πολύ πιθανό οι γείτονές της να ανήκουν σε διαφορετικά κομμάτια του συστήματος αρά και να μην έχουν σχέση μεταξύ τους.



Εικόνα 14 Clustering Coefficient and God Classes

Η Εικόνα 15 επιβεβαιώνει ότι η God Classes έχουν υψηλή σύζευξη και δείχνει πως είναι πολύ πιθανό για κάποια αλλαγή σε μια από τις κεντρικές κλάσεις να χρειαστούν πολλές αλυσόδετες αλλαγές και στις υπόλοιπες.



Εικόνα 15 Coupling και God Classes

Όλες οι παραπάνω παρατηρήσεις μπορούν να επιβεβαιωθούν και από τον πηγαίο κώδικα της κλάσης. Παρατηρούμε ότι υπάρχει μεγάλο πλήθος μεταβλητών δηλωμένων ως τύπο άλλης κλάσης, χρήση τέτοιων μεταβλητών ως παραμέτρους, χρήση των δεδομένων τούς αλλά και ύπαρξη πολλών μεθόδων με πολλές επαναλήψεις και μεγάλο βαθμό φωλιάσματος. Μια μέθοδος που παρουσιάζει αρκετές από τις παραπάνω παρατηρήσεις είναι η `setDestinationValue`.

```
private void setDestinationValue(MappingContextImpl<?, ?> context,
    MappingContextImpl<Object, Object> propertyContext, MappingImpl mapping,
    Converter<Object, Object> converter) {
    Object destination = context.getDestination();
    List<Mutator> mutatorChain = (List<Mutator>) mapping.getDestinationProperties();
    StringBuilder destPathBuilder = new StringBuilder();
    destPathBuilder.append(context.destinationPath);

    for (int i = 0; i < mutatorChain.size(); i++) {
        Mutator mutator = mutatorChain.get(i);
        destPathBuilder.append(mutator.getName()).append('.');
        String destPath = destPathBuilder.toString();

        // Handle last mutator in chain
        if (i == mutatorChain.size() - 1) {
            // Final destination value
            Object destinationValue = null;

            if (converter != null) {
                // Obtain from accessor on provided destination
                if (context.providedDestination) {
                    Accessor accessor = TypeInfoRegistry.typeInfoFor(destination.getClass(),
configuration)
                        .getAccessors()
                        .get(mutator.getName());
                    if (accessor != null) {
                        Object intermediateDest = accessor.getValue(destination);
                        propertyContext.setDestination(intermediateDest, true);
                    }
                }

                destinationValue = convert(propertyContext, converter);
            } else if (propertyContext.getSource() == null) {
                converter = converterFor(propertyContext);
                if (converter != null)
                    destinationValue = convert(propertyContext, converter);
            } else
                destinationValue = map(propertyContext);

            context.destinationCache.put(destPath, destinationValue);
            if (destinationValue != null || !configuration.isSkipNullEnabled())
                mutator.setValue(destination,
                    destinationValue == null ? Primitives.defaultValue(mutator.getType())
                        : destinationValue);
        }
    }
}
```

```

        if (destinationValue == null)
            context.shadePath(propertyContext.destinationPath);
    } else {
        // Obtain from cache
        Object intermediateDest = context.destinationCache.get(destPath);

        if (intermediateDest != null) {
            mutator.setValue(destination, intermediateDest);
        } else {
            // Obtain from circular destinations
            if (!context.intermediateDestinations.isEmpty()) {
                for (Object intermediateDestination : context.intermediateDestinations) {
                    // Match intermediate destinations to mutator by type
                    if (intermediateDestination.getClass().equals(mutator.getType())) {
                        intermediateDest = intermediateDestination;
                        mutator.setValue(destination, intermediateDest);
                        break;
                    }
                }
            }

            if (intermediateDest == null) {
                // Obtain from accessor on provided destination
                if (context.providedDestination) {
                    Accessor accessor = TypeInfoRegistry.typeInfoFor(destination.getClass(),
                        configuration)
                        .getAccessors()
                        .get(mutator.getName());
                    if (accessor != null)
                        intermediateDest = accessor.getValue(destination);
                }

                // Obtain from new instance
                if (intermediateDest == null) {
                    if (propertyContext.getSource() == null)
                        return;

                    intermediateDest =
createDestinationViaGlobalProvider(context.parentSource(),
mutator.getType(), context.errors);
                    if (intermediateDest == null)
                        return;

                    mutator.setValue(destination, intermediateDest);
                }
            }

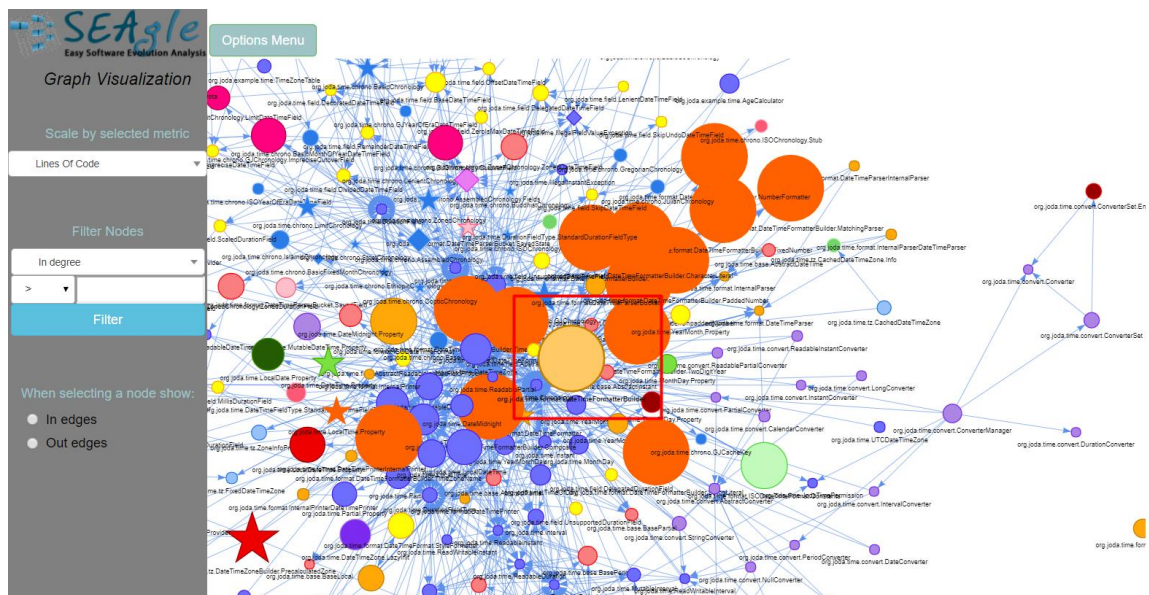
            context.destinationCache.put(destPath, intermediateDest);
        }

        destination = intermediateDest;
    }
}
}

```

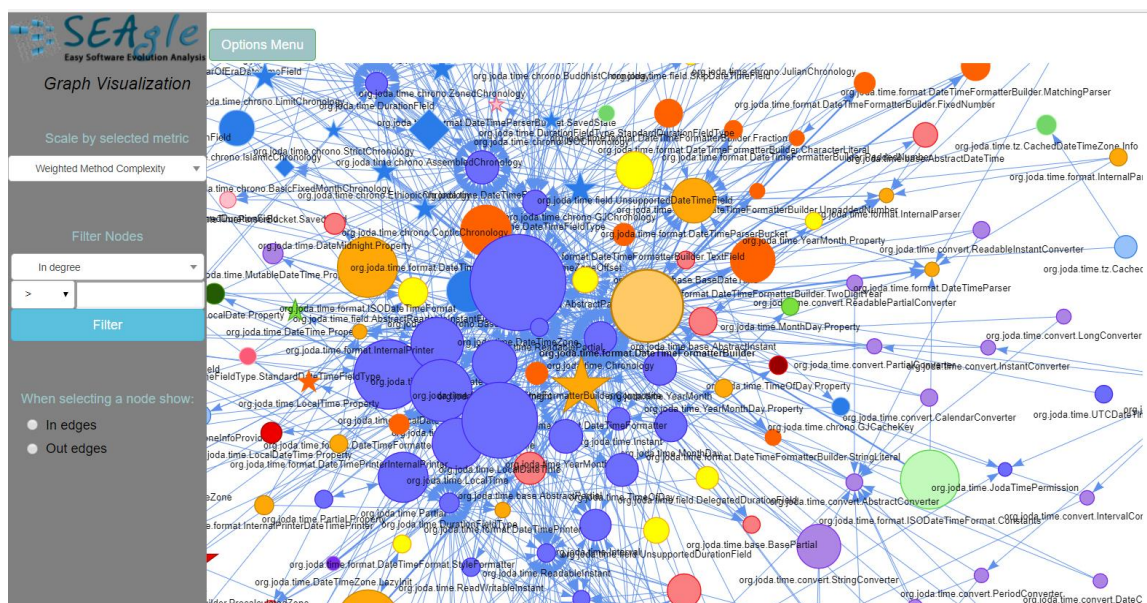

5.1.2 Εντοπισμός Brain Classes

Οι συγκεκριμένου τύπου κλάσεις μοιάζουν σε μεγάλο βαθμό με τις God Classes. Έχουν πολύ υψηλή πολυπλοκότητα, χαμηλή συνεκτικότητα και είναι μεγάλες (Εικόνα 16).



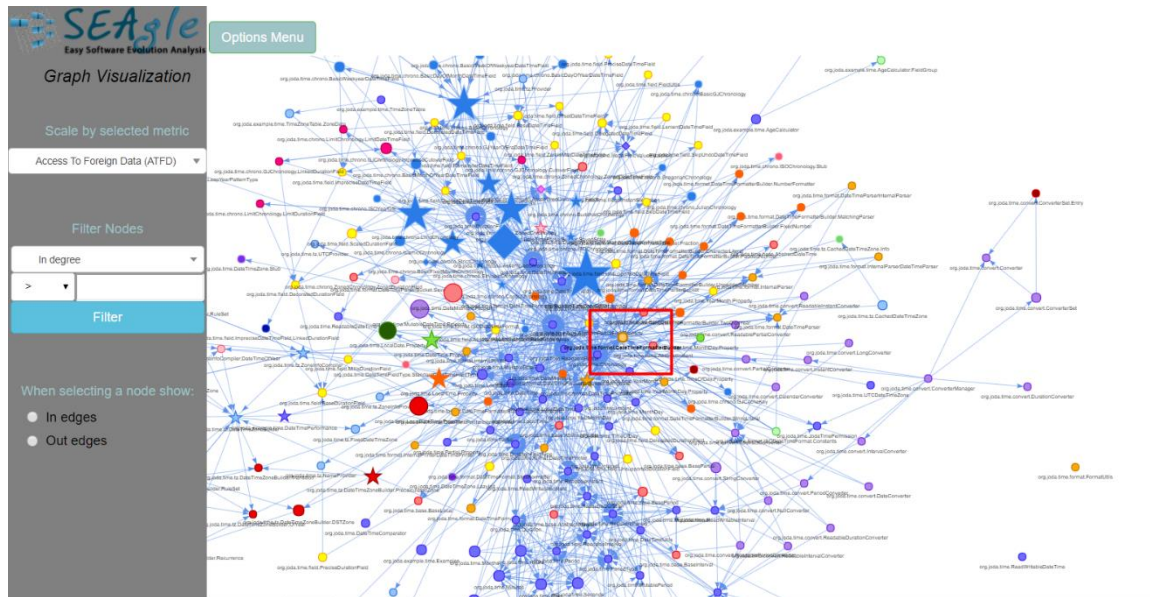
Εικόνα 16 : Lines of Code

Οι διπλανές πορτοκαλί κλάσεις μπορεί να είναι μεγάλες από άποψη γραμμών αλλά έχουν πολύ χαμηλή πολυπλοκότητα (Εικόνα 17)



Εικόνα 17 : WMC and Brain Classes

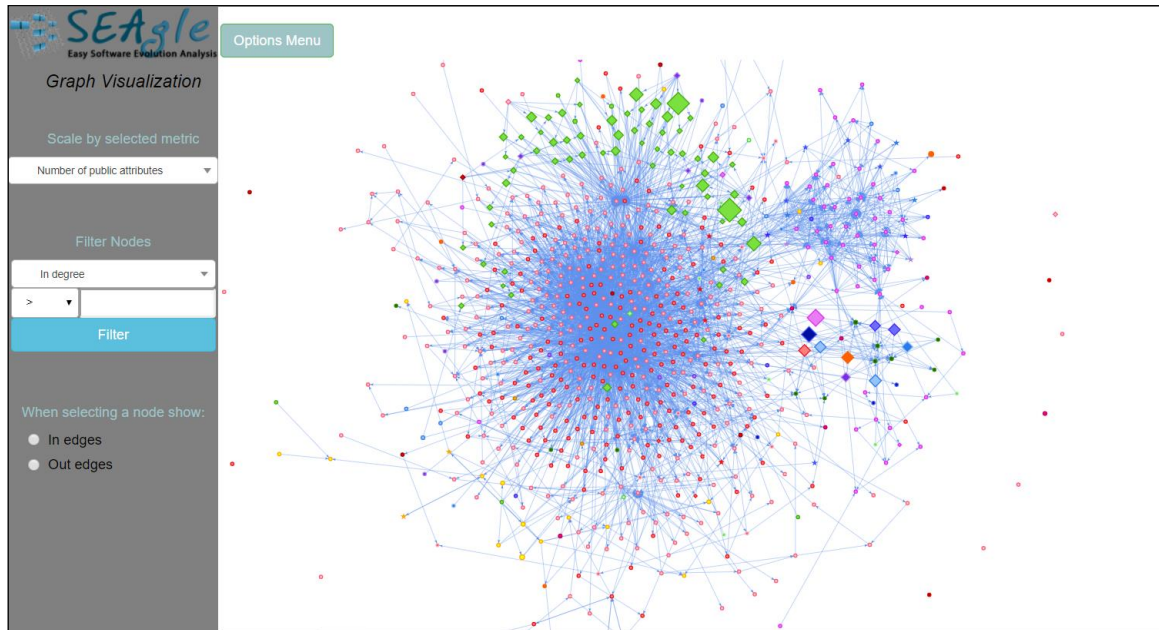
Η κυρία όμως διαφορά μεταξύ Brain και God Classes είναι η τάση τους να μην κάνουν κατάχρηση στην χρήση ξένων δεδομένων, αυτό γίνεται αντιληπτό και από την Εικόνα 18 στην οποία η κλάση μας έχει πολύ μικρή τιμή σε αντίθεση με τις διπλανές God Classes που ξεχωρίζουν.



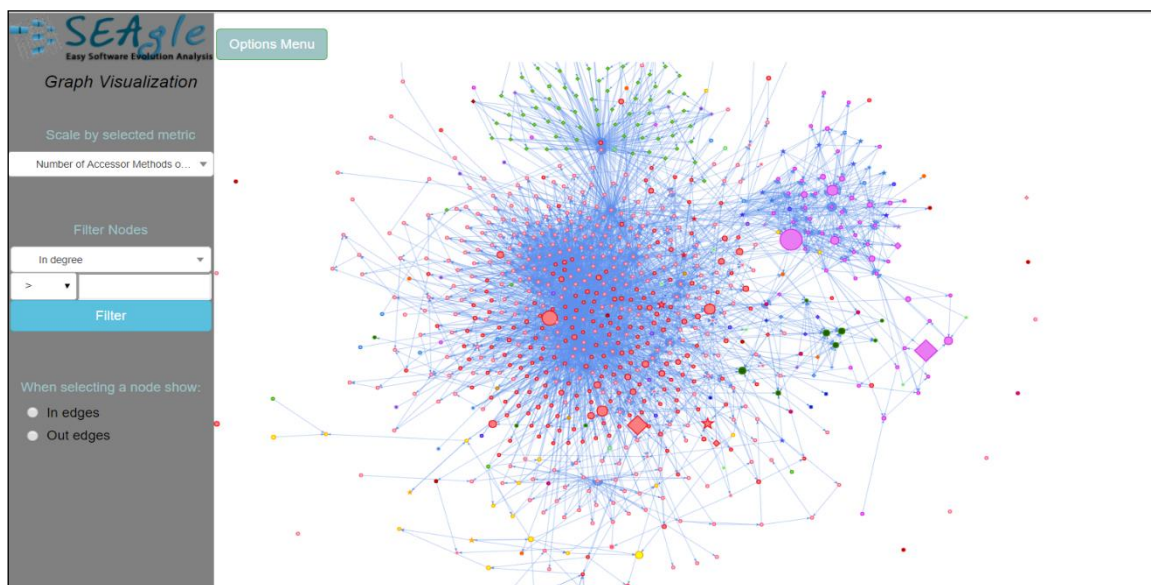
Εικόνα 18 : ATFD και Brain Classes

5.1.3 Εντοπισμός Data Classes

Το κύριο μέλημα των συγκεκριμένων κλάσεων είναι να προσφέρουν δεδομένα για το σύστημα χωρίς να συνεισφέρουν ουσιαστικά στην λογική του. Κυρία χαρακτηριστικά τους είναι η ύπαρξη μεγάλου πλήθους δημοσίων ιδιοτήτων σε αυτές (Εικόνα 19) αλλά και αυξημένος αριθμός Getter μεθόδων (Εικόνα 20).



Εικόνα 19 : NOPA και Data Classes



Εικόνα 20 : NOAM και Data Classes

5.2 Περίπτωση Χρήσης

Αρχικά δίνεται η δυνατότητα στον χρήστη να αναζητήσει το project που τον ενδιαφέρει από ένα select μενού. Το συγκεκριμένο μενού εμπεριέχει όλα τα java projects στα οποία έχει γίνει ανάλυση από την online πλατφόρμα Seagle. Όταν ο χρήστης ανοίγει το site η λίστα γεμίζει με το όνομα και το url των αναλυμένων projects. Η αναζήτηση του project μπορεί να γίνει και πληκτρολογώντας τα αρχικά γράμματα του ονόματος του ώστε να περιοριστούν τα ευρήματα (Εικόνα 22).

Μόλις γίνει η επιλογή του project που μας ενδιαφέρει τότε με βάση το όνομα του project που διάλεξε παραπάνω ο χρήστης γεμίζουμε το dropdown μενού με τα versions που σχετίζονται μαζί του. Επιλέγοντας το version που ενδιαφέρει τον χρήστη εμφανίζεται το κουμπί “Show Version’s Graph” όπου μας μεταφέρει στην σελίδα όπου γίνεται το visualization του γράφου για το συγκεκριμένο project και το συγκεκριμένο version (Εικόνα 23).



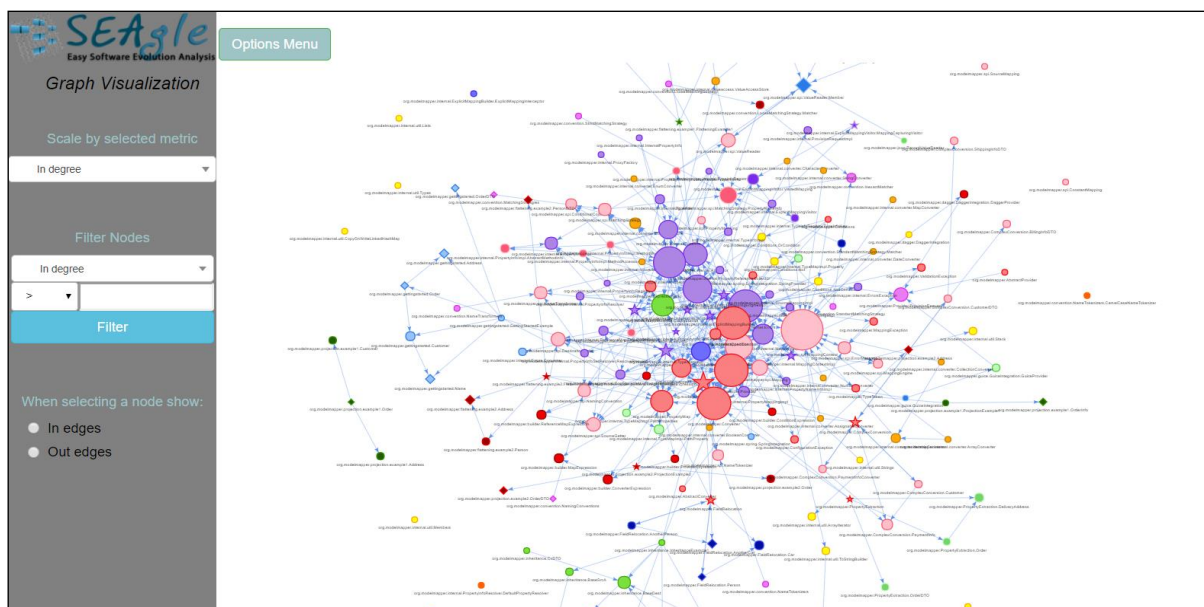
The screenshot displays the SEagle (Easy Software Evolution Analysis) Graph Visualization Site. At the top, the SEagle logo is shown with the tagline 'Easy Software Evolution Analysis'. Below the logo, the text 'Graph Visualization Site' is centered. A section titled 'Select project' contains a dropdown menu. The dropdown is open, showing a search input field with the text 'md'. Below the input field, three project entries are listed, each with its name and URL: 'Name = modelmapper URL = https://github.com/modelmapper/modelmapper.git', 'Name = commons-lang URL = https://github.com/apache/commons-lang.git', and 'Name = evolutionDemo URL = https://github.com/teohalk/evolutionDemo.git'. The first entry is highlighted with a blue background.

Εικόνα 22 : Αρχική σελίδα για αναζήτηση project



Εικόνα 23 : Επιλεγμένο project και γενιά

Πατώντας το κουμπί “Show Version’s Graph” εμφανίζεται η σελίδα όπου περιέχει τον γράφο του project και την πλαϊνή μπάρα για τις διάφορες επιλογές (Εικόνα 24).



Εικόνα 24 : Γραφική Αναπαράσταση Επιλεγμένου Project και Μπάρα Ρυθμίσεων

Ο γράφος που γίνεται visualize αποτελείται από του κόμβους που αναπαριστούν τις κλάσεις του java project και τα τόξα που συνδέουν του κόμβους σύμφωνα με το Law of Demeter. Περισσότερα σχετικά με τον γράφο στο [Κεφ 2.4.2](#) .

Η αρχική αναπαράσταση του γράφου γίνεται από πολύχρωμους κόμβους όπου το χρώμα τους εξαρτάται από τον parent folder ώστε να φαίνονται καλύτερα οι σχέσεις μεταξύ των κλάσεων.

Τα σχήματα των κόμβων που μπορεί να υπάρχουν είναι 4 και είναι τα εξής:

- Κύκλος : για τις κλάσεις όπου δεν έχουν συμπεριληφθεί κατά την διαδικασία της ανάλυσης σε κάποια κατηγορία Code smell.
- Αστέρι : για τις God Classes.
- Ρόμβος : για τις Data Classes.
- Τετράγωνο : για τις κλάσεις που παρουσιάζουν Feature Envy.

Αρχικά το scaling του γράφου γίνεται με την τιμή του in degree που έχει ο κάθε κόμβος, δηλαδή του πλήθους των τόξων που έχουν ως προορισμό τον συγκεκριμένο κόμβο.

Η πλαϊνή μπάρα είναι ένα μενού επιλόγων που προσφέρει την δυνατότητα στον χρήστη να επιλέξει την μετρική με την οποία θέλει να κάνει scaling τον γράφο, την δυνατότητα να φιλτράρει τους κόμβους σύμφωνα με τις τιμές τους σε οποιαδήποτε μετρική διαλέξει και ακόμη μπορεί να ρυθμίσει το τι θα συμβαίνει με την επιλογή ενός συγκεκριμένου κόμβου.

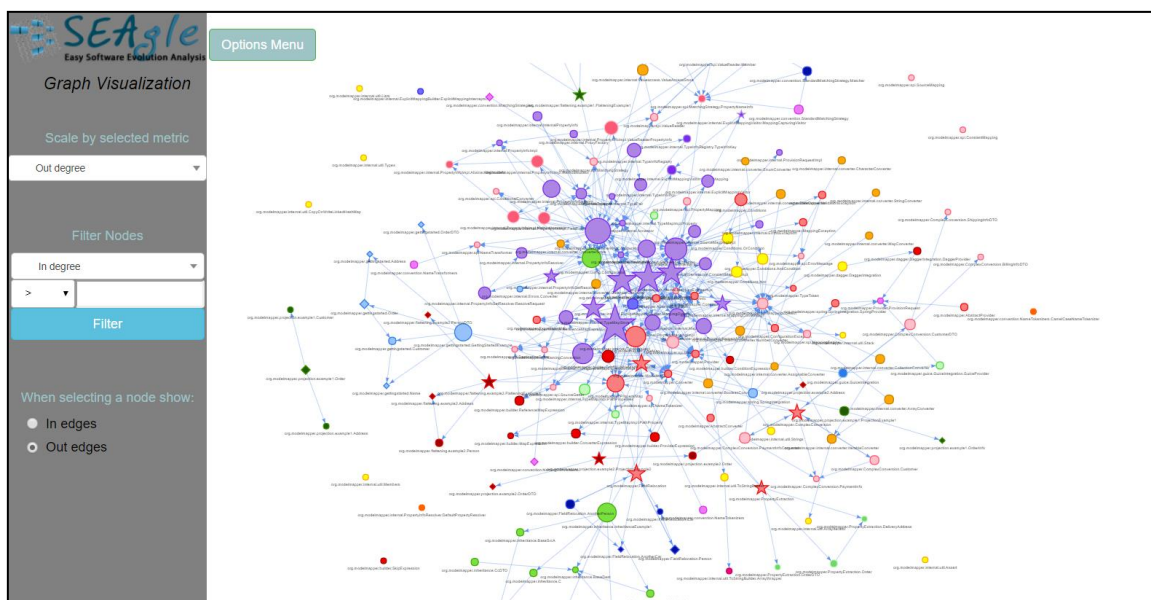
Η μετρικές όπου μπορεί να διαλέξει ο χρήστης για το scaling αλλά και για το φιλτράρισμα των κόμβων είναι οι εξής :

- In Degree
- Out Degree
- Clustering Coefficient
- Access To Foreign Data (ATFD)
- Betweenness Centrality

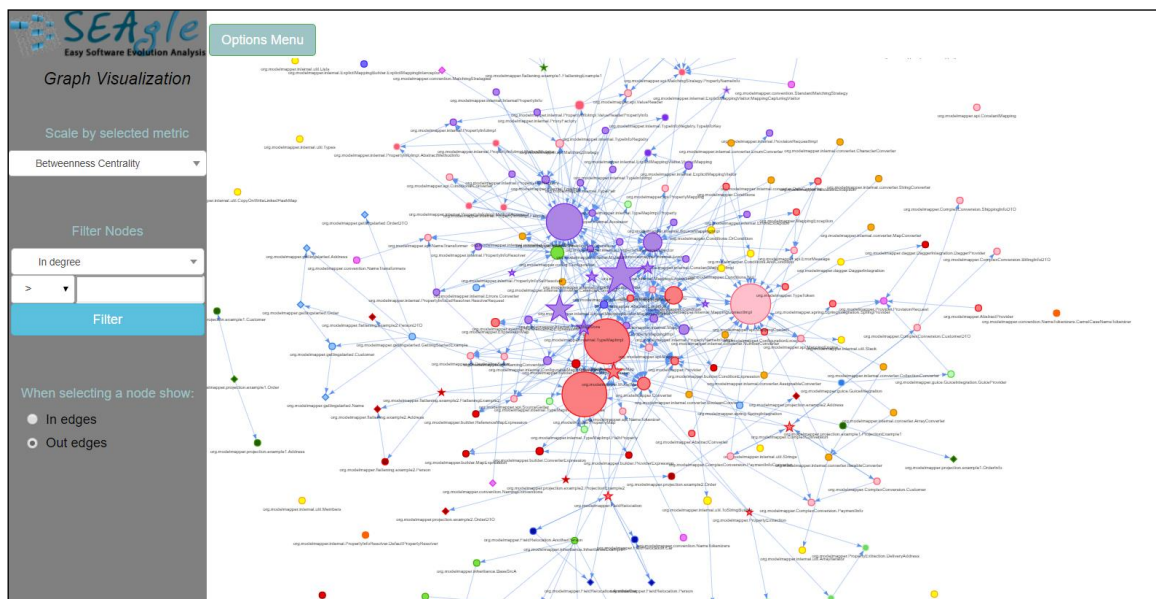
- Tight Class Cohesion (TCC)
- Weighted Method Complexity (WMC)
- Coupling Between Objects (CBO)
- Lack Of Cohesion in Methods 2 (LCOM2)
- Lines Of Code (LOC)
- Number of Accessor Methods of a Class (NOAM)
- Number of Methods (NOM)
- Number of Public Attributes (NOPA)
- Weight of Class (WOC)
- Locality of Attribute Access (LAA)
- Foreign Data Provider

Περισσότερα για κάθε μετρική στο [Κεφ 2.3](#).

Στο πρώτο drop down μενού ο χρήστης μπορεί να αναζητήσει την μετρική που τον ενδιαφέρει και επιλέγοντας την ο γράφος αλλάζει αυτόματα ώστε οι κόμβοι του να έχουν μέγεθος ανάλογο με την τιμή τους στην επιλεγμένη μετρική (Εικόνα 25 και Εικόνα 26) .



Εικόνα 25 : Scale με επιλεγμένη μετρική Out Degree



Εικόνα 26 : Scale με επιλεγμένη μετρική Betweenness Centrality

Λίγο πιο κάτω βρίσκεται μια ομάδα από drop down menus, input text και ενός κουμπιού.

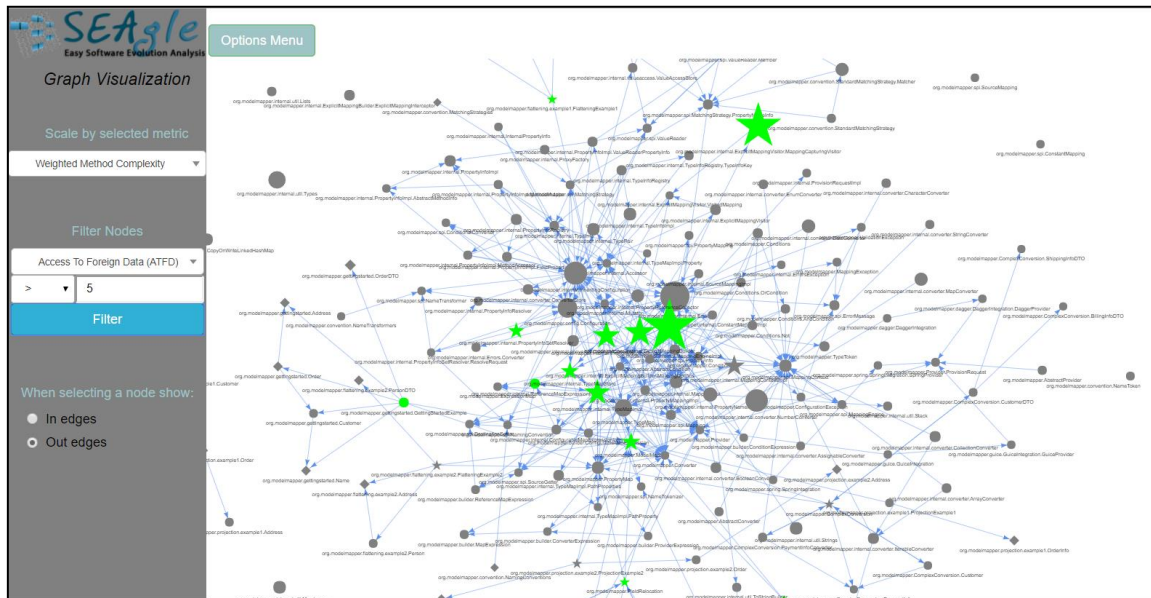
Ο ρόλος τους είναι να βοηθήσουν τον χρήστη να φιλτράρει τους κόμβους που περιέχονται στον γράφο. Η διαδικασία για το φιλτράρισμα είναι απλή.

Αρχικά ο χρήστης επιλέγει από το πρώτο drop down menu την μετρική όπου θέλει σύμφωνα με αυτή να φιλτραριστούν οι κόμβοι.

Στην συνέχεια διαλέγει αν θέλει να ψάξει για μεγαλύτερες (>) , μικρότερες (<) ή ίσες (=) τιμές συγκριτικά με την τιμή που εισάγει στο διπλανό input text.

Τέλος μόλις έχει ρυθμίσει τις παραπάνω παραμέτρους μπορεί να πατήσει το κουμπί “Filter” και θα δει να αλλάζουν τα χρώματα των κόμβων σε πράσινο και γκρι. Το πράσινο είναι για τους κόμβους όπου πληρούν τις προϋποθέσεις όπου έχει βάλει παραπάνω ο χρήστης , ενώ το γκρι για όσους κόμβους δεν τις πληρούν.

Ένα τέτοιο παράδειγμα βλέπουμε παρακάτω (Εικόνα 27). Ο χρήστης έχει επιλέξει να φιλτράρει τους κόμβους με βάση την τιμή τους στην μετρική ATFD (Access To Foreign Data) και συγκεκριμένα ζητάει όλους τους κόμβους με τιμή πάνω από 5. Το scaling που έχει επιλεγεί είναι η μετρική WMC (Weighted Method Complexity) που σχετίζεται με την πολυπλοκότητα της κλάσης.

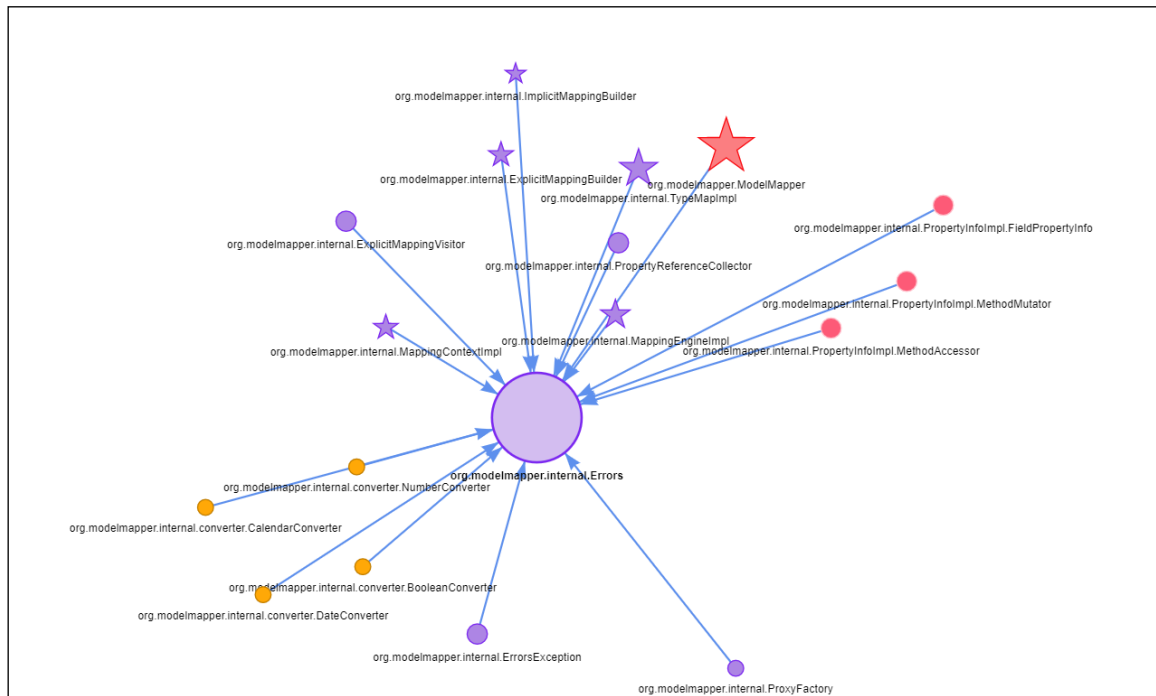


Εικόνα 27 : Φίλτρο για την μετρική ATFD και scale με WMC

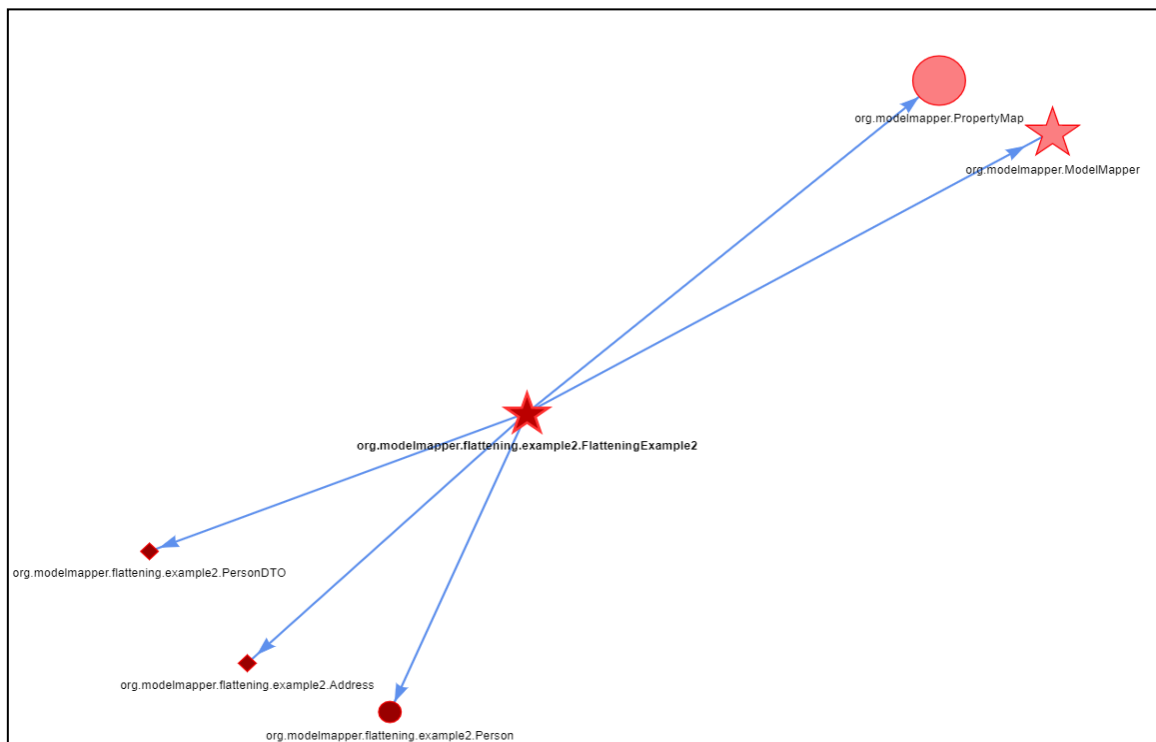
Ακόμη υπάρχουν δυο radio buttons όπου δίνουν την επιλογή στον χρήστη να διαλέξει την συμπεριφορά του γράφου όταν θα επιλέξει έναν συγκεκριμένο κόμβο.

Επιλέγοντας το πρώτο radio button με ετικέτα “In Degree” σημαίνει ότι όταν θα γίνει επιλογή ενός κόμβου θα εξαφανιστούν όλοι οι κόμβοι εκτός από αυτούς όπου συνδέονται μαζί του με τόξο με κατεύθυνση προς τον συγκεκριμένο κόμβο (Εικόνα 28).

Ενώ επιλέγοντας το radio button με ετικέτα “Out Degree” θα εξαφανιστούν πάλι όλοι οι κόμβοι εκτός από αυτούς όπου συνδέονται με τον επιλεγμένο κόμβο με τόξο που προέρχεται από τον ίδιο (Εικόνα 29).



Εικόνα 28 : Εμφάνιση μόνο των εισερχόμενων ακμών



Εικόνα 29 : Εμφάνιση μόνο των εξερχόμενων ακμών

6 Επίλογος

6.1 Σύνοψη

Η συγκεκριμένη εργασία ασχολήθηκε με την οπτική αναπαράσταση αντικειμενοστρεφών έργων ανοιχτού λογισμικού και το πώς μπορούμε να εξάγουμε πληροφορίες από την ανάλυση και την σωστή διαχείριση των μετρικών τους. Έγινε παρουσίαση των δυνατοτήτων της ηλεκτρονικής εφαρμογής όπου ο χρήστης θα μπορεί να επιλέγει για οπτικοποίηση σε μορφή γράφο το αναλυμένο έργο του και να τον επεξεργάζεται. Μέσω αυτών των δυνατοτήτων είδαμε πώς μπορούμε να εντοπίσουμε προβλήματα κακής σχεδίασης όπως είναι οι God classes , Brain Classes , Data Classes , Feature Envy αλλά και άλλες αποκλίσεις από τις αρχές αντικειμενοστρεφούς σχεδίασης.

6.2 Μελλοντική επέκταση

Τα μελλοντικά σχέδια εμπεριέχουν τόσο προσθήκες πάνω στο τωρινό κορμό του έργου όσο και εξερεύνηση και εφαρμογή του πάνω σε διαφορετικές πτυχές της ανάπτυξης λογισμικού. Αρχικά θέλουμε να κάνουμε ευκολότερη την σύνδεση μεταξύ του γράφου και του πηγαίου κώδικα. Για τον λόγο αυτό θα δώσουμε την δυνατότητα στον χρήστη με την επιλογή μια συγκεκριμένης κλάσης – κόμβου να αναδύεται η περιγραφή της σε UML και στην συνέχεια να μπορεί να μεταφερθεί στο αντίστοιχο κομμάτι στον πηγαίο κώδικα. Ακόμη θεωρούμε ότι εμφανίζει ενδιαφέρον η μελέτη και η γραφική απεικόνιση της αλληλεξάρτησης των διαφόρων bugs σε ένα σύστημα λογισμικού. Μία χαρτογράφηση ίσως τον σημείων όπου ένα bug επηρεάζει και συμπεράσματα για τους παράγοντες μέσω των οποίων γίνεται αυτή η σύνδεση.

Η έκρηξη ενδιαφέροντος για έργα ανοιχτού λογισμικού όπου συνεπάγεται σε μεγάλο πλήθος προγραμματιστών για το κάθε έργο θεωρούμε ότι παρουσιάζει ενδιαφέρον ώστε να μελετηθεί το πώς σχετίζονται μεταξύ τους οι προγραμματιστές για μία γενιά αλλά και την σύνδεση τους κατά την εξέλιξη ενός έργου. Παραδείγματος χάριν θα μπορούσε να φαίνεται για μία κλάση από ποιους προγραμματιστές αναπτύχθηκε, με ποια σειρά και πώς συνδέονται με τους υπόλοιπους.

Τέλος ένας ακόμα στόχος είναι η ανάλυση και οπτικοποίηση των έργων να μην περιορίζεται μόνο σε έργα γραμμένα σε Java αλλά να επεκταθεί και να τροποποιηθεί αναλόγως και για άλλες γλώσσες προγραμματισμού όπως είναι η δημοφιλέστατη Javascript.

Βιβλιογραφία

- [1] “Leonard Euler’s Solution to the Konigsberg Bridge Problem | Mathematical Association of America.” [Online]. Available: <https://www.maa.org/press/periodicals/convergence/leonard-eulers-solution-to-the-konigsberg-bridge-problem>. [Accessed: 06-Jan-2018].
- [2] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, 1 edition. Upper Saddle River, NJ: Prentice Hall, 2008.
- [3] A. J. Riel, *Object-Oriented Design Heuristics*, 1 edition. Place of publication not identified: Addison-Wesley Professional, 1996.
- [4] M. Fowler, K. Beck, J. Brant, W. Opdyke, D. Roberts, and E. Gamma, *Refactoring: Improving the Design of Existing Code*, 1 edition. Boston: Addison-Wesley Professional, 1999.
- [5] Robert C. Martin, “Design Principles and Design Patterns.” [Online]. Available: https://www.goodreads.com/work/best_book/45834062-design-principles-and-design-patterns.
- [6] R. C. Martin, *Agile Software Development, Principles, Patterns, and Practices*, 1st edition. Upper Saddle River, N.J: Pearson, 2002.
- [7] M. Lanza, R. Marinescu, and S. Ducasse, *Object-Oriented Metrics in Practice: Using Software Metrics to Characterize, Evaluate, and Improve the Design of Object-Oriented Systems*, Softcover reprint of hardcover 1st ed. 2006 edition. Berlin; London: Springer, 2010.
- [8] J. M. Bieman and B.-K. Kang, “Cohesion and Reuse in an Object-oriented System,” in *Proceedings of the 1995 Symposium on Software Reusability*, New York, NY, USA, 1995, pp. 259–262.
- [9] S. R. Chidamber and C. F. Kemerer, “A metrics suite for object oriented design,” *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, Jun. 1994.
- [10] B. Henderson-Sellers, *Object-Oriented Metrics: Measures of Complexity*, 1 edition. Upper Saddle River, N.J: Prentice Hall, 1995.
- [11] L. Erlikh, “Leveraging legacy system dollars for e-business,” *IT Prof.*, vol. 2, no. 3, pp. 17–23, May 2000.
- [12] J. Pavelka, G. Tel, and M. Bartosek, *SOFSEM’99: Theory and Practice of Informatics: 26th Conference on Current Trends in Theory and Practice of Informatics, Milovy, Czech Republic, November 27 - December 4, 1999 Proceedings*. Springer, 2003.
- [13] H. A. Muller and K. Klashinsky, “Rigi: a system for programming-in-the-large,” in *Proceedings. [1989] 11th International Conference on Software Engineering*, 1988, pp. 80–86.
- [14] M. Lanza, *The Evolution Matrix: Recovering Software Evolution using Software Visualization Techniques*. 2001.
- [15] M. Lanza and S. Ducasse, “Polymetric views - a lightweight visual approach to reverse engineering,” *IEEE Trans. Softw. Eng.*, vol. 29, no. 9, pp. 782–795, Sep. 2003.
- [16] R. Wettel and M. Lanza, “Visualizing Software Systems as Cities,” in *2007 4th IEEE International Workshop on Visualizing Software for Understanding and Analysis*, 2007, pp. 92–99.
- [17] R. Wettel and M. Lanza, “Visually Localizing Design Problems with Disharmony Maps,” in *Proceedings of the 4th ACM Symposium on Software Visualization*, New York, NY, USA, 2008, pp. 155–164.

- [18] T. Panas, R. Berrigan, and J. Grundy, “A 3D metaphor for software production visualization,” in *Proceedings on Seventh International Conference on Information Visualization, 2003. IV 2003.*, 2003, pp. 314–319.